

UNIVERSITY OF WOLVERHAMPTON
School of Mathematics and Computer Science



**Satellite-Based Land Cover Classification Using Deep Learning and Big
Earth Observation Data**

Submitted by
Prabin Upadhayaya
Student Number: 2439640

Supervisor
Emily Rodden

Date of Submission
18 December 2025

**A dissertation submitted in partial fulfilment of the requirements for the award of
the degree of**
Master of Science in Data Science
University of Wolverhampton

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me throughout the development of this project. First and foremost, I extend my deepest appreciation to my supervisor, **Emily Rodden**, whose guidance, constructive feedback, and continuous encouragement were invaluable at every stage of this research. Their insights greatly strengthened the quality and depth of this work.

I am profoundly grateful to my family for their unwavering support, patience, and belief in my academic journey. Their encouragement has been a constant source of motivation. Special thanks also go to my friends and colleagues who offered practical help, shared ideas, and provided reassurance during the challenging phases of experimentation and model development.

I would also like to acknowledge the developers and maintainers of the open-source tools and datasets used in this study, including ESA Sentinel-2, ESA WorldCover, PyTorch, Rasterio, and Scikit-learn. Their contributions made the implementation of this research possible.

Finally, I extend my gratitude to all faculty members and peers within my programme whose discussions, suggestions, and academic environment enriched my learning experience. This project would not have been possible without the collective support of all these individuals.

ABSTRACT

Land-cover classification plays a critical role in environmental monitoring, land management, and sustainable planning. With the increasing availability of high-resolution satellite imagery, deep learning methods—particularly convolutional neural networks—have emerged as powerful tools for semantic segmentation in Earth observation. This study investigates the feasibility and performance of a U-Net deep learning model for pixel-wise land-cover classification using multispectral Sentinel-2 Level-2A imagery and ESA WorldCover labels. An end-to-end processing pipeline was designed and implemented, comprising Sentinel-2 band extraction, spectral normalization, label reprojection, patch generation, model training, and full-scene inference. Due to hardware constraints, the data was downsampled, the number of valid labelled patches was limited, and several iterative methodological adjustments were required, including modifying patch extraction thresholds, correcting class index mismatches, and reducing batch sizes.

A Random Forest classifier was also implemented as a classical machine learning baseline for comparative analysis. Results indicate that both U-Net and Random Forest models achieved very low accuracy, with full-scene accuracy below 1%, near-zero IoU scores, and limited discriminative performance. Visual outputs also displayed extensive over-smoothing and class collapse. These findings highlight that model performance was fundamentally constrained by data limitations—including down sampling, label sparsity, and severe class imbalance—rather than by shortcomings of the deep learning architecture itself. Despite the low quantitative accuracy, the research successfully demonstrates a functional, reproducible segmentation pipeline and provides important insights into the methodological and computational challenges of remote-sensing deep learning under constrained conditions. The study concludes with detailed recommendations for improving data quality, expanding training samples, adopting advanced architectures, and utilizing higher-capacity computational resources in future work.

Contents

ACKNOWLEDGEMENT	i
ABSTRACT.....	ii
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	2
1.2.1 Data Heterogeneity	2
1.2.2. Lack of Labeled Ground-Truth	2
1.2.3. Poor Model Generalization Across Regions	2
1.2.4. Computational Limitations	2
1.2.5. Integration Challenges with EO Platforms	2
1.3 Research Aim	3
1.4 Research Objectives.....	3
1.5 Research Questions	3
1.6 Significance of the Study	3
1.6.1. Advancement of Deep Learning in Remote Sensing	4
1.6.2. Scalable and Reproducible EO Workflows.....	4
1.6.3. Improved Land Cover Accuracy	4
1.6.4. Contribution to Open Science	4
1.6.5. Application Potential in Developing Regions.....	4
1.7 Novelty of the Study	4
1.8 Scope and Limitations.....	5
CHAPTER 2: LITERATURE REVIEW	6
2.1 Introduction.....	6
2.2 Global and Regional Land-Cover Products	6
2.2.1 Early global land-cover maps	6
2.2.2 Regional land-cover products	6
2.2.3 High-resolution global maps.....	7
2.3 Sentinel-2 for Land-Cover Mapping.....	7
2.3.1 Sentinel-2 mission and spectral characteristics.....	7
2.3.2 Preprocessing and atmospheric correction.....	7
2.3.3 Sentinel-2 in land-cover and land-use studies	8
2.4 Classical Machine Learning for Land-Cover Classification.....	8

2.4.1 Overview of traditional classifiers	8
2.4.2 Random Forests in remote sensing	9
2.5 Deep Learning for Semantic Segmentation of Remote-Sensing Imagery	9
2.5.1 CNNs and fully convolutional networks.....	9
2.5.2 U-Net and encoder–decoder architectures	9
2.5.3 DeepLab and other advanced segmentation models	10
2.5.4 Semantic segmentation for land-cover mapping.....	10
2.6 ESA WorldCover as Training Data and Weak Supervision	11
2.6.1 Product characteristics	11
2.6.2 Limitations and accuracy	11
2.6.3 WorldCover as weak labels for deep learning.....	11
2.7 Label Noise, Class Imbalance and Patch Sampling.....	12
2.7.1 Label noise in EO datasets	12
2.7.2 Class imbalance	12
2.7.3 Patch-based learning	12
2.8 Lightweight Architectures and Resource-Constrained Pipelines.....	13
2.9 Evaluation Metrics for Segmentation and Land-Cover Mapping.....	13
2.10 Summary	14
CHAPTER 3: RESEARCH GAP	15
3.1 Introduction.....	15
3.2 Limitations of Global Land-Cover Products for Local Applications.....	15
3.3 Under-Explored Use of Lightweight U-Net with Sentinel-2 and WorldCover.....	16
3.4 Limited Fair Comparisons with Random Forest Baselines Under Identical Constraints	16
3.5 Incomplete Practical Treatment of Label Noise and Patch Sampling.....	17
3.6 Lack of Transparent, Reproducible, Resource-Aware Pipelines for Education and Practice	17
3.7 Summary of Research Gap and Link to Objectives.....	18
CHAPTER 4: METHODOLOGY	20
4.1 Introduction.....	20
4.2 Study Area and Data Sources.....	20
4.3 Preprocessing of Sentinel-2 Imagery	20
4.3.1 Ideal Preprocessing Workflow	20
4.3.2 Practical Constraints and Adjustments (Actual Implementation).....	21
4.3.3 Band Normalisation	21
4.3.4 Output and Verification.....	21
4.4 Label Preparation and Alignment with WorldCover.....	22

4.4.1 Use of WorldCover Data.....	22
4.4.2 Reprojection and Resampling.....	22
4.4.3 Handling Label Mismatch and Unlabeled Pixels.....	22
4.5 Patch Extraction	22
4.5.1 Ideal Approach.....	22
4.5.2 Actual Practical Adaptation.....	23
4.6 U-Net Architecture.....	23
4.6.1 Architectural Overview.....	23
4.6.2 Rationale	23
4.7 Training Procedure.....	23
4.7.1 Training Settings	23
4.7.2 Practical Constraints	23
4.8 Random Forest Baseline	23
4.8.1 Ideal Setting	23
4.8.2 Actual Working Setup	24
4.9 Evaluation Metrics	24
CHAPTER 5: IMPLEMENTATION	25
5.1 Configuration and Reproducibility	25
5.2 Band Discovery, Reading, and Stacking.....	25
5.2.1 Band discovery.....	25
5.2.2 Downsampled reading	25
5.2.3 Resampling 20 m bands	26
5.2.4 Band stacking.....	26
5.3 Label Alignment and Class Remapping.....	26
5.3.1 Alignment.....	26
5.3.2 Remapping labels.....	26
5.4 Patch Extraction and Dataset Construction.....	27
5.4.1 Patch extraction.....	27
5.4.2 Patch dataset.....	27
5.5 U-Net Model Architecture	27
5.6 Training Procedure and Loss / Accuracy Monitoring	28
5.6.1 Training loop.....	28
5.7 Random Forest Baseline Implementation	28
5.8 Full-Scene Inference and Predicted Map Generation	29
5.9 Visualisation of Model Outputs	29

5.10 Summary	30
CHAPTER 6: RESULT AND DISCUSSION	31
6.1 Overview	31
6.2 Visual Inspection of Input Data and Preprocessing Outputs	31
6.3 Full Scene Prediction Analysis	32
6.4 Patch-Based Evaluation Results	33
6.5 Random Forest Baseline Results	34
6.6 Discussion of Key Factors Affecting Performance.....	34
6.6.1 Downsampling Effects	34
6.6.2 Label Misalignment	35
6.6.3 Class Imbalance	35
6.6.4 Limited Training Patches	35
6.6.5 Comparison with Literature	35
6.7 Summary of Findings.....	36
6.8 Conclusion	36
CHAPTER 7: CONCLUSION AND FUTURE WORK	37
7.1 Introduction.....	37
7.2 Summary of Research Objectives and Approach.....	37
7.3 Key Findings from the Implemented Pipeline	38
7.3.1 Functional End-to-End Deep Learning Pipeline.....	38
7.3.2 Quantitative Performance: Low Accuracy but High Diagnostic Value	38
7.3.3 Qualitative Analysis: Over-smoothing and Class Collapse	38
7.3.4 Strong Influence of Data and Implementation Constraints.....	40
7.4 Critical Reflection on Methodology and Implementation	40
7.5 Future Work	41
7.5.1 Use Multiple Sentinel-2 Tiles and Larger Training Sets.....	41
7.5.2 Train at Full 10 m Resolution Using Better Hardware	41
7.5.3 Improve Class Balancing and Loss Design	41
7.5.4 Evaluate More Advanced Architectures.....	41
7.5.5 Enhance Label Quality and Ground-Truth Sources	41
7.5.6 Extend Evaluation and Visual Analytics	42
7.6 Final Remarks	42
REFERENCES	44
APPENDICES	46

List of Figures:

Figure 1: Normalised and Stacked Sentinel-2 Multispectral Composite.....	21
Figure 2 : Stacked Multispectral Sentinel-2 Image (Normalized)	31
Figure 3: Full Scene U-Net Land Cover Prediction.....	32
Figure 4 : Example Patch-wise Land Cover Prediction.....	33
Figure 5: Random Forest Classification Report.....	34
Figure 6 : Qualitative analysis of Model.....	39
Figure 7 : Metrics Evaluation	42

CHAPTER 1: INTRODUCTION

1.1 Background

Land cover classification plays a critical role in understanding, managing, and monitoring the Earth’s surface. As global environmental concerns intensify—including climate change, deforestation, agricultural expansion, urbanization, biodiversity loss, and land degradation—the need for accurate, real-time land cover mapping has never been more pressing [3][8][41]. Land cover information informs decision-making in fields such as environmental conservation, disaster response, food security planning, hydrology, urban development, carbon accounting, and protected-area management [23][40]. Traditional classification techniques, however, have long struggled to match the scale, resolution, and complexity of modern environmental dynamics [6][17][33].

The last two decades have witnessed rapid advancements in satellite-based Earth Observation (EO). Missions such as the European Space Agency’s Sentinel-2, NASA’s Landsat program, and commercial constellations have dramatically increased access to high-resolution multispectral imagery. Sentinel-2, in particular, provides systematic global coverage with 10–20 m resolution across 13 spectral bands, making it suitable for fine-grained land cover monitoring [11][15][44]. Parallel to this, cloud platforms such as Google Earth Engine (GEE), Amazon Web Services (AWS), and Microsoft Planetary Computer have enabled large-scale data access and geospatial computation [27].

Despite the abundance of EO data, extracting meaningful insights remains challenging. Traditional algorithms such as Maximum Likelihood Classifier, K-Means, Support Vector Machines (SVM), and Random Forests depend on handcrafted features and often fail to generalize across geographical regions or multisensory datasets [17][29]. They also struggle with spectral confusion, high intraclass variability, atmospheric distortions, and temporal inconsistencies.

Deep learning has emerged as an effective solution to many of these limitations. Convolutional Neural Networks (CNNs), fully convolutional networks, U-Net, DeepLab architectures, and Transformer-based models have revolutionized remote sensing, achieving state-of-the-art performance in semantic segmentation and pixel-level classification [2][10][19][30]. Deep learning’s ability to extract hierarchical features directly from imagery makes it highly suitable for complex tasks such as differentiating croplands from grasslands, shrublands from forests, or built-up areas from bare soil—categories that often share overlapping spectral signatures [26].

The integration of deep learning with large EO datasets marks a transformative moment for land cover mapping. However, several critical challenges remain, including limited annotated data, generalization difficulties across landscapes, and high computational demands when processing continental-scale imagery[14][41]. To address these issues, this research explores an end-to-end pipeline combining Sentinel-2 imagery, ESA WorldCover ground truth labels, big-data preprocessing (GEE-like workflows), and a PyTorch-based U-Net deep learning architecture. The aim is to produce a scalable, accurate, and reproducible model for land cover classification—improving over traditional classifiers while demonstrating practical applicability in real environmental monitoring workflows.

1.2 Problem Statement

Although deep learning has demonstrated superior performance in remote sensing tasks, its adoption in large-scale EO workflows remains limited due to substantial technical barriers: [27][41]

1.2.1 Data Heterogeneity

Satellite images are influenced by a range of factors, including atmospheric conditions such as clouds, haze, and shadows, seasonal phenological variations, sensor noise, differences in spatial resolution, and topographic effects. Together, these factors introduce variability and uncertainty in the observed spectral signatures, which can significantly reduce classification accuracy, particularly when working with multisource or multitemporal datasets [11][23][40].

1.2.2. Lack of Labeled Ground-Truth

High-quality, pixel-level annotated datasets such as ESA WorldCover, BigEarthNet, and EuroSAT are widely used in remote sensing applications; however, they present several limitations. These datasets often suffer from class imbalance, contain a certain degree of label noise, and may include temporal mismatches between the imagery and reference labels[1][12]. In addition, spatial autocorrelation within patches can bias model learning, while the datasets do not fully capture the diversity of global landscapes. Furthermore, the provided annotations may not align perfectly with Sentinel-2 pixel geometry, which can introduce additional uncertainty during training and evaluation [22][31].

1.2.3. Poor Model Generalization Across Regions

Most published studies achieve high accuracy locally but fail to generalize to new geographic areas. As highlighted in multiple papers deep learning models often overfit to specific biomes or countries and struggle when applied globally [29][36].

1.2.4. Computational Limitations

Training convolutional neural networks on large Sentinel-2 mosaics requires substantial computational resources, including access to GPUs, [28] image tiling strategies, memory-efficient preprocessing pipelines, and cloud-based scaling solutions. These requirements increase system complexity and cost, which can limit both reproducibility of experiments and the practicality of operational deployment, particularly in resource-constrained research or production environments [16][38].

1.2.5. Integration Challenges with EO Platforms

While platforms such as Google Earth Engine (GEE) enable efficient large-scale preprocessing of satellite imagery, they lack native support for deep learning model training and optimisation. As a result, researchers must rely on external frameworks and data transfers, which increases the complexity of developing truly end-to-end pipelines and introduces additional challenges related to integration, reproducibility, and workflow automation [35].

Overall, there is a clear need for a scalable, transferable, and accurate deep learning–based framework that can process Sentinel-2 data in an end-to-end manner. Such a framework should seamlessly integrate cloud-based preprocessing, support the training of robust land cover segmentation models, produce high-quality land cover maps, and enable rigorous performance evaluation against strong baseline methods such as Random Forest classifiers. This thesis aims to address these challenges by proposing a reproducible and automated workflow that bridges the gap between cloud-based geospatial preprocessing and deep learning model development.

1.3 Research Aim

The primary aim of this research is to develop an end-to-end deep learning pipeline using Sentinel-2 imagery and Earth Observation big data for accurate, multi-class land cover classification, leveraging a U-Net model and cloud-scale preprocessing workflows.[19][20]

1.4 Research Objectives

To achieve the research aim, the following objectives are established:

Objective 1: Preprocess Sentinel-2 imagery using cloud-based workflows, including:

- cloud masking,
- radiometric normalization,
- band stacking,
- resampling (20 m $\rightarrow$ 10 m),
- label alignment using ESA WorldCover.

Objective 2: Develop and train a U-Net model in PyTorch for pixel-level segmentation.

Objective 3: Generate and use high-quality image patches for robust model training, incorporating image augmentation strategies.

Objective 4: Evaluate model performance using standard metrics:

- Intersection over Union (IoU),
- F1-score,
- confusion matrix,
- per-class accuracy.

Objective 5: Compare U-Net performance against a traditional Random Forest classifier.

Objective 6: Assess the generalization potential of the model across geographic regions and landscapes.

Objective 7: Produce a reusable and scalable code pipeline that automates the entire workflow end-to-end.

1.5 Research Questions

This thesis seeks to answer the following research questions:

1. How effectively can a deep learning-based U-Net architecture classify land cover from Sentinel-2 multispectral imagery compared to traditional machine learning approaches?
2. What preprocessing steps are essential for reliable, large-scale, and automated land cover classification using EO data?
3. What is the performance difference between CNN-based segmentation and Random Forest classifier when trained on identical datasets?
4. How well does the proposed model generalize to unseen regions and heterogeneous landscapes?
5. Can the end-to-end pipeline be feasibly integrated into operational satellite data workflows?
[2][6][29][33][43]

1.6 Significance of the Study

The significance of this research is multi-dimensional:

1.6.1. Advancement of Deep Learning in Remote Sensing

This study presents a complete deep learning pipeline that is broadly applicable across multiple real-world domains, including environmental monitoring, smart agriculture, land-use planning, and the development of climate adaptation strategies, demonstrating the versatility and practical relevance of the proposed framework [2][26].

1.6.2. Scalable and Reproducible EO Workflows

The proposed pipeline incorporates Google Earth Engine–style preprocessing, PyTorch-based model training, and tile-based inference strategies, which together ensure scalability and efficient processing when applied to large-scale satellite datasets.

1.6.3. Improved Land Cover Accuracy

CNN-based architectures such as U-Net generally outperform traditional classifiers, and this study empirically validates these advantages through improved Intersection over Union (IoU) scores, better spatial consistency in predicted maps, and reduced classification noise.

1.6.4. Contribution to Open Science

All code components mimic or integrate with open EO data sources (Sentinel-2, WorldCover). This supports transparent, repeatable research.[13][16]

1.6.5. Application Potential in Developing Regions

Many regions, including Nepal, Ethiopia, and Sub-Saharan Africa, lack up-to-date high-resolution land cover maps. This framework can support:

- forest monitoring,
- agricultural planning,
- biodiversity conservation,
- early warning systems.

1.7 Novelty of the Study

The novelty of this work lies in:

- 1. Integrating full Sentinel-2 SAFE decoding**
 - ✓ Many studies use preprocessed tiles; this pipeline reads raw SAFE archives.
- 2. Automated multi-band stacking with 10 m resampling**
 - ✓ Aligns 10 m and 20 m bands into dense inputs for U-Net.
- 3. End-to-end training pipeline using PyTorch**
 - ✓ Includes augmentation, tiling, patch extraction.
- 4. Benchmark comparison with Random Forest baseline**
 - ✓ Demonstrates quantifiable advantages of deep learning.
- 5. Scalable inference across full satellite scenes**
 - ✓ Uses tile-based predictions and raster stitching.
- 6. Application of ESA WorldCover alignment with dynamic reprojection**
 - ✓ Ensures pixel-perfect training labels.

These components make the pipeline reproducible, adaptable, and operationally relevant.

1.8 Scope and Limitations

Scope

This study focuses on pixel-level land cover classification using Sentinel-2 optical multispectral imagery, with ESA WorldCover employed as the labeled ground truth. The proposed framework addresses an 11-class semantic segmentation problem and is designed to operate over study areas covering diverse and configurable geographic regions [1]. The performance of a deep learning-based U-Net model is systematically compared against a traditional Random Forest classifier to evaluate the effectiveness of deep learning for large-scale land cover mapping.[14]

- Pixel-level land cover classification
- Sentinel-2 optical multispectral imagery
- ESA WorldCover as labeled ground truth
- 11-class segmentation problem
- Configurable study areas with diverse geographic characteristics
- Comparison of U-Net and Random Forest models

Limitations

Despite its effectiveness, the proposed approach has several limitations. Cloud contamination may still negatively affect model accuracy, and the reliance on Sentinel-2 optical data means that SAR-related information is not captured, limiting performance in dense forests or complex urban environments [16]. Model generalization is influenced by the diversity and representativeness of the training patches, while the computational demands of deep learning training make GPU resources preferable. In addition, ESA WorldCover labels are subject to noise and are not perfectly accurate. These limitations are acknowledged and addressed where possible within the study [27][35].

- Residual cloud effects impacting accuracy
- Lack of SAR information in Sentinel-2 data
- Dependence on training data diversity for generalization
- High computational requirements for model training
- Presence of label noise in ESA WorldCover

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

Land-cover classification is a core component of environmental monitoring, biodiversity conservation, agricultural management, hydrological modelling and climate policy. High-quality land-cover maps underpin decisions ranging from forest protection and peatland restoration to urban planning and disaster risk reduction. Traditionally, these maps were derived from moderate-resolution satellite sensors using relatively simple classification techniques. Over the last decade, however, three developments have reshaped the field:[9][40][44]

1. The emergence of high-resolution, freely available satellite data such as Sentinel-2;[3]
2. The availability of global land-cover products at 10–30 m resolution, such as ESA WorldCover; and[25]
3. The rapid adoption of machine learning and deep learning, especially convolutional neural networks (CNNs), for semantic segmentation of remote-sensing imagery. [20][26]

This chapter reviews the literature relevant to the work presented in this project, which develops a lightweight U-Net-based land-cover classification pipeline using Sentinel-2 imagery and ESA WorldCover labels, and benchmarks its performance against a Random Forest baseline. The review covers global and regional land-cover datasets, the characteristics of Sentinel-2 data and its use in land-cover classification, and classical machine-learning approaches—particularly Random Forests—as well as recent advances in deep learning for semantic segmentation and land-cover mapping, with a specific focus on U-Net architectures [10][24]. In addition, the role of ESA WorldCover as a training or validation data source is examined, alongside key challenges such as label noise, class imbalance, and spatial or temporal mismatches between imagery and reference labels.

The literature review further considers the development of resource-constrained and lightweight segmentation pipelines suitable for scalable and operational use. These themes are explicitly linked to the design choices adopted in this project, including spatial downsampling, label remapping, patch extraction, the use of a compact U-Net architecture, and benchmarking against a Random Forest classifier. By establishing these connections, the chapter provides a clear conceptual foundation that grounds the methodology and analysis presented in subsequent chapters [35][44].

2.2 Global and Regional Land-Cover Products

2.2.1 Early global land-cover maps

Early global land-cover maps relied on sensors such as AVHRR, SPOT-VEGETATION and MODIS, which offered frequent revisit times but relatively coarse spatial resolutions (250–1,000 m). Products such as GLC2000, MODIS MCD12Q1 and GlobCover represented major advances by providing harmonised, global classifications. However, their coarse resolution limited their suitability for applications that require fine-scale information, such as mapping small agricultural fields, narrow riparian zones, or peri-urban mosaics.[41]

2.2.2 Regional land-cover products

In parallel, several regional products were developed, including CORINE Land Cover (CLC) for Europe and national land-cover maps for the UK, USA and other countries. These often provide more detailed class legends, but still face issues such as:

- heterogeneous methods across countries or years;
- limited temporal resolution; and
- variable thematic accuracy depending on land-cover type.

Such products remain widely used as references for validation and change detection but are increasingly complemented by higher-resolution mapping.[8][25][40][41]

2.2.3 High-resolution global maps

With advances in computing and the availability of high-resolution EO data, global land-cover maps at 30 m and 10 m have emerged. Examples include GlobeLand30, CCI Land Cover at 300 m, and more recently ESA WorldCover at 10 m. These products leverage sensors such as Landsat-8 and Sentinel-2 alongside machine-learning algorithms to produce more detailed global maps.[3][12][22]

WorldCover 2020/2021 is especially relevant to this project: it provides 11 land-cover classes at 10 m resolution and is derived from Sentinel-1 and Sentinel-2 imagery using advanced classification methods. Studies evaluating WorldCover generally find good performance at broad scales but note class-specific and regional limitations, especially for spectrally similar vegetation and mosaicked landscapes. This creates a natural motivation to refine or adapt WorldCover locally using targeted models trained on original Sentinel-2 data.[40][41]

2.3 Sentinel-2 for Land-Cover Mapping

2.3.1 Sentinel-2 mission and spectral characteristics

The Sentinel-2 mission, part of the European Copernicus programme, consists of two identical satellites (Sentinel-2A and 2B) carrying the MultiSpectral Instrument (MSI). MSI provides:

- 13 spectral bands from the visible (VIS) to shortwave infrared (SWIR);
- 10 m resolution in Blue (B02), Green (B03), Red (B04) and NIR (B08);
- 20 m resolution in four red-edge bands and SWIR bands B11–B12;
- a swath width of 290 km; and
- revisit time of around 5 days at the equator (with both satellites operating).

These characteristics distinguish Sentinel-2 from older missions such as Landsat by offering finer spatial resolution, more spectral bands tailored to vegetation (red-edge), and a higher revisit frequency. The 10 m bands are particularly important for land-cover mapping: B02–B04 capture visible reflectance, while B08 measures near-infrared reflectance strongly related to vegetation biomass and canopy structure.[3][11]

Shortwave infrared bands B11 and B12 provide information on soil and vegetation moisture, mineral content and burned areas, and are often included in classification and burned-area studies. Since these have 20 m resolution, they are commonly resampled to 10 m (using bilinear or cubic methods) to match the grid of B02–B04 and B08 when training pixel-wise models.[34]

2.3.2 Preprocessing and atmospheric correction

Sentinel-2 Level-1C products provide top-of-atmosphere reflectance. For land-surface applications, these are typically converted to bottom-of-atmosphere (BOA) reflectance using atmospheric correction tools such as Sen2Cor. Level-2A products, which this project uses, already include BOA reflectance and some quality information, such as a Scene Classification Layer (SCL) that assigns classes like cloud, shadow, vegetation and bare ground. Cloud and snow masking are often critical preprocessing steps.[38]

In practice, the degree of preprocessing depends on the study region and cloud conditions. Some land-cover studies rely heavily on multi-temporal compositing and cloud filtering, whereas others operate on single scenes with minimal cloud contamination, as in this project.[15][23]

2.3.3 Sentinel-2 in land-cover and land-use studies

A substantial body of research demonstrates the effectiveness of Sentinel-2 imagery for land-cover classification. Large-scale surveys of Sentinel-2 land monitoring applications highlight its strong performance and widespread adoption across a variety of land-cover mapping tasks [3]. In particular, time-series-based approaches have shown that incorporating multi-temporal Sentinel-2 data improves classification accuracy by capturing seasonal dynamics and temporal patterns in land-cover change [15]. In addition, studies focusing on spectral feature extraction and band engineering have demonstrated that the rich multispectral information provided by Sentinel-2 enhances class separability and leads to improved classification performance [11][34].

More recent research has emphasized the role of advanced machine-learning and deep-learning methods for Sentinel-2-based land-cover mapping. Deep-learning approaches, including ensemble models and semantic segmentation pipelines, have achieved strong results at regional to global scales [2][4][26][36]. Comparative studies further indicate that while classical machine-learning methods such as Random Forest remain strong baselines for Sentinel-2 classification, deep-learning models often provide superior spatial consistency for pixel-level mapping [5][6][29]. Building on these findings, this project stacks Sentinel-2 bands B02, B03, B04, B08, B11, and B12 into a multi-band raster, which serves as the input for both the proposed U-Net model and the Random Forest baseline [13][34].

2.4 Classical Machine Learning for Land-Cover Classification

2.4.1 Overview of traditional classifiers

Before the widespread adoption of deep learning, land-cover classification predominantly relied on so-called shallow machine-learning classifiers. Early approaches included the Maximum Likelihood Classifier (MLC), which is grounded in probabilistic decision theory and was widely used in initial land-cover studies, particularly with moderate-resolution multispectral imagery [17][29]. Distance-based methods such as k-Nearest Neighbours (k-NN) were also commonly applied, assigning land-cover classes based on spectral similarity in feature space [17][29]. Support Vector Machines (SVMs) later gained prominence by separating classes in high-dimensional feature spaces and demonstrated strong performance even with limited training samples in Sentinel-2 and Landsat-based classification tasks [17][29]. In addition, ensemble learning methods such as Random Forests (RF) emerged as competitive alternatives by combining multiple decision trees to improve robustness and classification accuracy when using multispectral satellite data [5][6][29][33].

SVMs were particularly popular in the early 2000s due to their effectiveness with small training datasets; however, they scale poorly to very large datasets and require careful tuning of kernel functions and regularization parameters. In contrast, Random Forests became increasingly favored because of their ability to model nonlinear relationships and interactions between spectral bands, their relative robustness to noise and overfitting, and their capacity to provide informative measures of variable importance. Moreover, Random Forests require minimal hyperparameter tuning, making them well suited as strong baseline models for land-cover classification in large-scale remote sensing applications [6][17].

2.4.2 Random Forests in remote sensing

Random Forests construct an ensemble of decision trees, each trained on a bootstrap sample of the training data, with random subsets of features selected at each split (Breiman, 2001). The final prediction is the majority vote (classification) or average (regression) of individual trees. RFs have been successfully applied to:

- general land-cover classification using Landsat and Sentinel-2 imagery [3,6,15,23,29];
- forest type mapping and biomass-related land monitoring [3,15,27,41];
- urban land-use / urban land-cover classification and fine-scale urban mapping [4,36,40]; and
- crop type discrimination and cropland mapping [5,33,15].

Many comparative studies have demonstrated that Random Forest (RF) classifiers often outperform traditional parametric approaches such as Maximum Likelihood Classification (MLC) and, in some cases, Support Vector Machines (SVM), particularly when the feature space comprises multiple spectral bands, indices, and temporal information [6,15,23,29]. Owing to its ensemble-based structure, RF is robust to noise, non-linearity, and high-dimensional feature spaces, making it well suited for multispectral Sentinel-2 land-cover classification tasks [6,27].

However, RF primarily operates on per-pixel feature vectors and does not explicitly model spatial neighbourhood relationships, which are critical for accurately capturing spatial patterns in heterogeneous landscapes [27,29]. To introduce spatial context, several studies augment RF inputs with texture features, morphological profiles, or object-based segmentation outputs, derived from multispectral imagery or pre-segmentation steps [26,27,33]. While these approaches can improve classification accuracy, they substantially increase preprocessing complexity and rely on manually engineered spatial descriptors [26].

In contrast, Convolutional Neural Networks (CNNs) inherently learn hierarchical spatial–spectral features through convolutional filters, enabling them to capture contextual information at multiple scales without explicit feature engineering [2,27,30]. This capability makes CNN-based architectures more flexible and effective for complex land-cover classification tasks, particularly in high-resolution Sentinel-2 imagery where spatial patterns play a crucial role [2,30,36].

In this project, a lightweight Random Forest baseline is trained on sampled labelled pixels from the same Sentinel-2 stack used by the U-Net. Sampling is capped (e.g. 3,000 points) and the number of trees is reduced to avoid memory errors, mirroring the resource constraints of the overall pipeline. The RF model serves as a classical baseline against which the performance and feasibility of the lightweight U-Net are assessed.

2.5 Deep Learning for Semantic Segmentation of Remote-Sensing Imagery

2.5.1 CNNs and fully convolutional networks

Deep CNNs have revolutionized computer vision tasks such as image classification, object detection, and segmentation [27]. For semantic segmentation, a key innovation was the development of fully convolutional architectures, which replace fully connected layers with convolutional operations to enable dense, pixel-wise predictions [26]. These models enabled the first wave of end-to-end segmentation networks in remote sensing applications [2].

2.5.2 U-Net and encoder–decoder architectures

U-Net, originally developed for biomedical image segmentation, consists of:

- an encoder path that progressively downsamples the image, extracting high-level features; and
- a decoder path that upsamples feature maps back to the original resolution.

Crucially, U-Net uses skip connections between encoder and decoder layers to recover fine spatial detail. This architecture has proven extremely successful for remote-sensing segmentation because it:

- handles variable-sized inputs via fully convolutional operations;
- works well with relatively small training datasets when combined with augmentation;
- captures both local detail and broader spatial context.[2][19][20][27]

U-Net and its variants have been applied to:

- building and road extraction from high-resolution aerial and satellite imagery;
- urban land-cover mapping at fine spatial scales;
- cropland and vegetation segmentation using multispectral data from Sentinel-2 and PlanetScope; and
- change detection and flood mapping in disaster monitoring scenarios [2,26,36,40].

Several extensions of the original architecture have been proposed to further enhance performance. U-Net++ introduces nested and dense skip connections to improve multi-scale feature fusion, while attention-based U-Net variants incorporate attention mechanisms to focus the model on relevant spatial regions [10,20]. However, these architectural enhancements typically increase memory consumption and computational cost, which can limit their applicability in large-scale or resource-constrained remote-sensing deployments [24].

2.5.3 DeepLab and other advanced segmentation models

Beyond U-Net, the DeepLab family of models introduces atrous (dilated) convolutions and Atrous Spatial Pyramid Pooling (ASPP) to capture rich multi-scale contextual information without excessive spatial downsampling [27]. These design choices allow DeepLab variants to effectively model objects at different spatial scales, and DeepLab v3+ has demonstrated strong performance on widely used semantic segmentation benchmarks [27].

More recently, transformer-based models and hybrid CNN–Transformer architectures have been introduced for remote-sensing image segmentation, leveraging self-attention mechanisms to model long-range spatial dependencies [30]. Such approaches have shown promising results for large-scale and complex land-cover mapping tasks [30].

However, these architectures are typically computationally demanding, requiring substantial GPU memory and long training times. In many MSc-level research projects or field-based operational settings, such requirements are often impractical. Consequently, there is a clear need for lightweight segmentation models that retain the spatial–contextual advantages of U-Net-style architectures while remaining feasible to train and deploy on modest computational hardware [24].

2.5.4 Semantic segmentation for land-cover mapping

In remote sensing, CNN-based segmentation methods have been widely applied to a range of land-cover mapping tasks. Fully convolutional and encoder–decoder architectures have been successfully used for land-cover classification by integrating very-high-resolution (VHR) and Sentinel-2 imagery, consistently demonstrating clear performance improvements over traditional Random Forest classifiers [2]. Large-scale CNN-based studies on aerial and satellite imagery have also highlighted practical challenges associated with tiling strategies, memory limitations, and GPU resource requirements when processing

high-resolution data over extensive spatial domains [27]. In addition, comprehensive reviews of Earth observation segmentation methods emphasise a clear paradigm shift from conventional pixel-based machine-learning approaches toward fully convolutional deep learning models capable of end-to-end semantic segmentation [27].

However, many of these studies implicitly assume access to large, well-curated training datasets and high-performance computing infrastructure. In contrast, this project deliberately targets a more constrained and realistic operational setting by focusing on a single Sentinel-2 tile, noisy ESA WorldCover reference labels, and a compact U-Net architecture. This design prioritises robustness, computational feasibility, and reproducibility under limited resource conditions, while still enabling effective pixel-level land-cover segmentation [12][40][44].

2.6 ESA WorldCover as Training Data and Weak Supervision

2.6.1 Product characteristics

ESA WorldCover is a global land-cover product with a spatial resolution of 10 m, initially released for the year 2020 and subsequently updated for 2021, providing 11 land-cover classes such as tree cover, shrubland, grassland, cropland, built-up areas, bare or sparse vegetation, wetlands, and water bodies [12]. The dataset is produced using Sentinel-1 and Sentinel-2 imagery as primary inputs and machine-learning-based classification pipelines to generate consistent global land-cover maps. Its key strengths include a higher spatial resolution compared to earlier global land-cover products, consistent class definitions across different regions, and global coverage that enables uniform land-cover analysis worldwide [44].

2.6.2 Limitations and accuracy

Independent assessments show that WorldCover has relatively good overall accuracy but exhibits class-specific and region-specific errors. Challenges include:

- confusion between spectrally similar vegetation types (e.g. shrubland vs grassland);
- misclassification of heterogeneous mosaics, particularly in agricultural landscapes;
- inaccuracies along boundaries (e.g. forest–cropland edges, narrow rivers).

These limitations imply that WorldCover should not be treated as perfect ground truth, especially for fine-scale local studies. Instead, it may be more appropriate to treat it as a noisy supervision source or as a baseline to be refined.[8][22]

2.6.3 WorldCover as weak labels for deep learning

An emerging line of research leverages ESA WorldCover and similar global land-cover products as weak supervision to train deep neural networks directly on Sentinel-2 imagery. The underlying rationale is that WorldCover offers dense pixel-level labels over very large areas, convolutional neural networks are capable of learning robust and transferable feature representations even in the presence of label noise, and trained models can subsequently be fine-tuned or adapted for local or regional applications [12]. Despite this potential, there remain relatively few well-documented examples of single-scene, tile-level pipelines that use WorldCover as training data for U-Net or comparable semantic segmentation architectures. This project helps to address this gap by implementing a practical and reproducible workflow that aligns WorldCover with a Sentinel-2 tile using WarpedVRT, remaps raw class codes into a compact set of class identifiers while reserving zero for unlabeled pixels, and integrates these remapped labels into a patch-based U-Net training framework [44].

- Use of WorldCover as weak supervision for Sentinel-2–based segmentation
- Pixel-aligned label preparation through dynamic reprojection and remapping
- Patch-based U-Net training on a single Sentinel-2 scene

2.7 Label Noise, Class Imbalance and Patch Sampling

2.7.1 Label noise in EO datasets

Label noise arises from misclassification in reference products, misalignment between imagery and labels, mixed pixels and temporal changes between acquisition dates. In remote sensing, label noise is almost unavoidable, especially when labels are derived from automated global products rather than manual annotation.[31]

Noisy labels can:

- reduce model performance and generalisation;
- mislead the training process, especially if errors are systematic; and
- bias evaluation metrics when reference maps are themselves inaccurate.

Approaches for handling label noise include the use of noise-robust loss functions, curriculum learning, label smoothing, and methods such as confident learning, which aim to identify and down-weight potentially corrupted labels during training [14,31]. In this project, a simpler and pragmatic strategy is adopted: unlabelled or clearly unreliable pixels are assigned a value of 0 and excluded from optimisation by setting `ignore_index = 0` in the cross-entropy loss. In addition, image patches with extremely low proportions of valid labelled pixels are discarded to reduce the influence of severe label noise during model training.

2.7.2 Class imbalance

Class imbalance is a well-known challenge in land-cover mapping, as classes such as built-up areas, wetlands, snow or ice, and certain crop types typically occupy much smaller spatial extents than dominant classes like cropland or grassland, leading models trained with naïve loss functions to overpredict majority classes while underrepresenting minority categories [27][40][41]. Common mitigation strategies include oversampling minority classes or undersampling majority classes, using class-balanced loss functions such as inverse-frequency weighting or median-frequency balancing, and applying focal loss to increase the influence of hard-to-classify samples during training [42]. In this project, the primary focus is on developing a robust and functional end-to-end pipeline rather than exhaustively optimising class-balanced loss formulations; however, the adopted patch sampling strategy, which enforces a minimum proportion of valid labelled pixels per patch, ensures that training samples contain meaningful land-cover information, reduces bias toward unlabeled or background-dominated regions, and partially alleviates the effects of class imbalance.

2.7.3 Patch-based learning

Due to memory constraints, many semantic segmentation pipelines operate on image patches rather than full scenes, particularly when dealing with high-resolution remote-sensing imagery [27]. In patch-based learning, images are divided into fixed-size windows (for example, 128×128 pixels), which are extracted using a sliding-window strategy—sometimes with overlap—and each patch is treated as an individual training sample paired with its corresponding label patch [35]. This approach offers several advantages, including a reduced memory footprint, the ability to shuffle patches to improve gradient diversity during training, and greater flexibility for data augmentation techniques such as flips and rotations.

In this project, the patch generator is designed with practicality and robustness in mind. It uses a user-defined patch size (e.g. 128×128 pixels), applies a stride equal to the patch size to generate non-overlapping patches for simplicity, and enforces a minimum valid fraction of labelled pixels (e.g. at least 1%) for a patch to be retained [17]. This strategy ensures that extracted patches provide non-trivial supervision, helping to maintain effective training even in the presence of noisy labels and under constrained computational resources.

2.8 Lightweight Architectures and Resource-Constrained Pipelines

Most state-of-the-art semantic segmentation models assume access to high-memory GPUs and large-scale computational infrastructure; however, in practice, many users—particularly students and practitioners in low-resource settings—work on desktop or laptop machines with limited RAM and, in some cases, CPU-only environments. This constraint motivates research into lightweight neural architectures and memory-aware data handling strategies [24]. Architectural solutions proposed in the literature include depthwise separable convolutions popularised by MobileNet architectures, inverted residual blocks for improved parameter efficiency, model pruning and quantisation to reduce memory and computational demands, and compact U-Net variants with fewer channels and reduced network depth [24][27]. Complementary data-handling strategies include spatial downsampling of imagery, conversion to lower-precision numerical formats such as float16, the use of very small batch sizes, and limiting the total number of training patches.

The UNetSmall model adopted in this project embodies these lightweight design principles. It uses fewer convolutional filters per layer (16–32–64), a shallow encoder–decoder structure, and is trained with small batch sizes on downsampled Sentinel-2 multi-band stacks (for example, using a scale factor of 0.125). This design enables full-scene training and inference on a single machine while preserving the essential encoder–decoder structure and skip connections that characterise standard U-Net architectures, thereby achieving a practical balance between computational feasibility and segmentation performance.

2.9 Evaluation Metrics for Segmentation and Land-Cover Mapping

Accuracy assessment is a central component of land-cover mapping and Earth observation analysis, with commonly used evaluation metrics including Overall Accuracy (OA), which measures the proportion of correctly classified pixels; Producer’s and User’s Accuracy, corresponding to per-class recall and precision; the F1-score, defined as the harmonic mean of precision and recall; and Intersection-over-Union (IoU), which quantifies the overlap between predicted and reference areas for each class, along with its class-averaged variant, mean IoU (mIoU) [40][41]. In deep semantic segmentation, mIoU has become the de facto standard evaluation metric and is widely adopted in benchmark datasets such as PASCAL VOC and Cityscapes, as IoU-based measures are more informative than OA in imbalanced classification settings where high accuracy can be achieved by predicting majority classes alone [27].

In this project, the U-Net model is evaluated using per-class IoU, mean IoU, and macro F1-score computed over all labelled pixels within the scene, while the Random Forest baseline is assessed using a standard classification report reporting precision, recall, and F1-score based on sampled labelled pixels. Model performance is interpreted not only through quantitative metrics but also through qualitative visual analysis by comparing predicted land-cover maps with the corresponding Sentinel-2 imagery and reference labels, enabling a more comprehensive assessment of spatial consistency and classification quality.

2.10 Summary

This chapter has synthesised the key literature underpinning the project and established a clear methodological foundation for the work that follows. Global land-cover products such as ESA WorldCover provide valuable 10 m resolution maps with worldwide coverage, yet they exhibit class-specific and region-dependent inaccuracies, particularly in complex and heterogeneous landscapes [44]. Sentinel-2 imagery, with its rich multispectral and spatial detail, has therefore emerged as a standard and highly effective data source for land-cover classification in Earth observation research [3][23]. While classical machine-learning approaches—most notably Random Forests—remain strong and widely used baselines, they are limited in their ability to explicitly model spatial context and spatial coherence in pixel-level predictions [6][29].

In contrast, deep-learning architectures, particularly U-Net and its variants, have fundamentally transformed semantic segmentation in remote sensing by jointly learning spatial and spectral representations in an end-to-end manner [2][27]. At the same time, the use of global land-cover products as weak supervision is gaining traction, offering a practical means of generating dense training labels at scale; however, well-documented implementations at the single-tile level under constrained computational resources remain relatively scarce [14][31]. Persistent challenges such as label noise, class imbalance, and limited hardware availability further motivate the need for careful label handling, patch-based training strategies, lightweight network architectures, and robust evaluation protocols.

Taken together, these insights justify the central focus of this project: the design, implementation, and analysis of a memory-aware U-Net pipeline for Sentinel-2 land-cover classification using ESA WorldCover labels, alongside a systematic comparison with a Random Forest baseline under identical data conditions. The remaining research gaps and specific objectives arising from this review are articulated in the following chapter, providing a clear transition from theory to methodology.

CHAPTER 3: RESEARCH GAP

3.1 Introduction

The literature review demonstrates that the core ingredients for advanced land-cover mapping—high-resolution Sentinel-2 imagery, global land-cover products such as ESA WorldCover, and modern deep-learning architectures—are now widely available [3,12,23,27]. Despite this progress, important gaps remain between state-of-the-art research prototypes, operational global products, and what can be realistically implemented in low-resource or MSc-level project settings [24,31].

This chapter therefore synthesises the specific research gaps that motivate the present work and explicitly links them to the methodological choices adopted in this project. These gaps can be grouped into five main areas, which are discussed in the following sections.

- i. Limitations of global products for local-scale applications;
- ii. Under-explored combination of Sentinel-2, WorldCover and lightweight U-Net;
- iii. Limited comparative studies with Random Forest under identical constraints;
- iv. Incomplete treatment of label noise and patch sampling in practical pipelines; and
- v. Lack of transparent, reproducible implementations for teaching and capacity building.

3.2 Limitations of Global Land-Cover Products for Local Applications

Global land-cover products such as ESA WorldCover are primarily designed to support analyses at continental to global scales. Although these products achieve high overall accuracies, several limitations emerge when they are applied to local or tile-scale analyses:

- Class confusion in heterogeneous landscapes: Spectrally similar vegetation classes—such as shrubland, grassland, and cropland—are frequently confused, particularly in fragmented or mosaicked agricultural regions where within-class variability is high [8,25,41].
- Boundary-related errors: Misclassification commonly occurs along class boundaries, including forest–cropland edges, narrow rivers, wetlands, and field margins, where mixed pixels, geolocation uncertainty, and resampling effects are prevalent [40,44].
- Temporal mismatch effects: WorldCover represents a fixed reference year, whereas Sentinel-2 scenes capture land surface conditions at specific acquisition dates. Land-use or phenological changes between these dates can introduce apparent label inconsistencies when WorldCover is used as ground truth [22,41].
- Legend rigidity and semantic mismatch: The fixed 11-class legend may be either overly coarse for local land-management applications or overly detailed in cases where class boundaries are not spectrally separable at 10 m resolution, leading to unstable or ambiguous labels [3,23].

Despite these limitations, global land-cover products are increasingly adopted as training labels for deep-learning models because they offer spatially exhaustive, globally consistent coverage that would otherwise be prohibitively expensive to generate manually [12,44]. However, there remains limited methodological guidance on how to systematically adapt, refine, or reinterpret WorldCover labels at tile scale, particularly when training lightweight deep-learning models directly on Sentinel-2 imagery under constrained computational resources.

3.3 Under-Explored Use of Lightweight U-Net with Sentinel-2 and WorldCover

Most deep-learning studies in land-cover mapping tend to fall into one of two extremes. On one hand, large-scale approaches employ high-capacity models—such as DeepLab variants, full U-Net, or U-Net++—trained on multiple tiles or entire regions using cloud-based platforms and powerful GPUs. On the other hand, smaller-scale studies often rely on patch-based classification using curated benchmark datasets such as EuroSAT, which provide clean labels and allow the use of relatively modest model architectures under controlled conditions [2]. While both approaches have advanced the field, they do not adequately reflect the constraints faced in many real-world academic and applied settings.

Notably, there is a lack of research explicitly investigating the combination of a single Sentinel-2 tile (or a small number of tiles), ESA WorldCover as a source of noisy training labels [44], a lightweight U-Net architecture tailored to limited memory environments, and a fully documented end-to-end Python/PyTorch pipeline capable of running on standard laptop or desktop hardware [35]. This gap is particularly relevant for MSc-level projects and small organisations, where practitioners typically have access to only a few local Sentinel-2 scenes, global land-cover products, and constrained computational resources. Consequently, the research gap addressed in this project is both technical and practical: how to design and implement a memory-aware semantic segmentation pipeline that operates within these constraints while still delivering meaningful and interpretable land-cover classification results.

The UNetSmall model and associated data-handling strategies proposed in this project directly target this gap by:

- reducing channel widths to minimise memory consumption;
- downsampling Sentinel-2 imagery where necessary;
- using lower-precision data types (e.g. half-precision arrays) when appropriate; and
- employing small batch sizes combined with patch-based sampling.

Together, these design choices demonstrate a feasible and reproducible approach to deep-learning-based land-cover segmentation under realistic hardware limitations.

3.4 Limited Fair Comparisons with Random Forest Baselines Under Identical Constraints

Random Forest (RF) classifiers remain widely used in operational land-cover mapping because of their robustness, interpretability, and computational efficiency [6]. Nevertheless, many deep-learning studies either omit RF baselines altogether or configure them in ways that hinder fair comparison—for example, by using different spatial resolutions, relying on hand-crafted features for RF while feeding raw imagery to convolutional neural networks, or training RF models on substantially smaller spatial extents than their deep-learning counterparts [29]. As a result, it is often unclear whether deep models genuinely offer a substantial advantage under realistic constraints, or whether simpler methods might perform comparably when trained and tuned on the same data.

A key gap addressed by this project is the establishment of a fair, resource-aware comparison between a compact U-Net trained on patch-based Sentinel-2 inputs using ESA WorldCover labels and a Random Forest model trained on the same multi-band stack and label source, with labelled pixels sampled and model complexity constrained to avoid memory limitations. By training and evaluating both approaches under identical data and hardware constraints, the project provides a realistic assessment of their relative

performance—even when absolute accuracy is limited by data quality—as well as the practical trade-offs in implementation complexity and the interpretability of results. This comparison offers valuable insights for practitioners seeking to determine whether the additional complexity of deep-learning approaches is justified in their specific operational or resource-constrained contexts.

3.5 Incomplete Practical Treatment of Label Noise and Patch Sampling

Academic research on label noise has progressed rapidly in recent years, proposing sophisticated techniques to identify, correct, or down-weight corrupted labels during training, including noise-robust loss functions and confident learning approaches [14,31]. In contrast, many practical segmentation pipelines in remote sensing continue to treat global reference products as if they were reliable ground truth, often applying only minimal filtering or masking [41].

In tile-based Sentinel-2 + WorldCover workflows, several practical challenges arise:

- large unlabeled or unreliable regions, where WorldCover is missing, masked, or clearly inconsistent with contemporaneous Sentinel-2 imagery;
- severely imbalanced class distributions, in which classes such as built-up areas, water bodies, or wetlands occupy only a small fraction of the scene [40]; and
- patch windows dominated by background or a single class, providing limited or misleading supervisory signal during training.

Many published codes either skip these details or assume a curated label mask. The gap here is practical guidance on how to:

- remap raw WorldCover integer codes to a compact set of labels suitable for training (keeping 0 as “ignore”);
- configure the loss function (e.g. `ignore_index=0` in cross-entropy);
- choose a minimum valid labelled fraction threshold for keeping patches; and
- integrate these design choices into a reproducible pipeline.

The patch-generation and label-remapping components of this project directly address this gap by documenting and implementing a concrete, end-to-end solution. Unlabelled or unreliable pixels are explicitly set to label 0, training patches are retained only if they satisfy a minimum proportion of valid labels, and optimisation explicitly ignores label 0 during loss computation. While this approach does not represent a state-of-the-art noise-robust learning method, it is straightforward, transparent, and reproducible, making it particularly suitable for MSc-level research and resource-constrained applied work, where clarity and reliability often outweigh algorithmic complexity.

3.6 Lack of Transparent, Reproducible, Resource-Aware Pipelines for Education and Practice

Finally, a more pedagogical gap exists in the remote-sensing deep-learning literature: many studies do not release complete codebases, or they provide implementations that assume access to cloud platforms or specialised computational infrastructure. This makes it difficult for students, conservation practitioners, and local agencies to fully understand the workflow from raw Sentinel-2 .SAFE products to final classified rasters, to adapt existing pipelines to new geographic regions or land-cover legends, or to experiment with different network architectures and hyperparameters on modest hardware.

This project partially addresses that gap by designing an end-to-end, modular, and resource-aware pipeline. The workflow spans the complete process from reading Sentinel-2 bands from a .SAFE directory,

stacking and normalising inputs, aligning ESA WorldCover labels using WarpedVRT, generating patches, training both a U-Net and a Random Forest model, performing full-scene inference, and computing evaluation metrics. Each major task is implemented as a separate, well-documented function, with key design decisions—such as downsampling, patch size selection, and label remapping—explicitly grounded in the literature and constraints discussed in Chapter 2. By presenting methodology and implementation in parallel, the project offers a practical blueprint that can be adapted to other regions or global land-cover products, contributing not only to scientific understanding but also to capacity building in geospatial deep learning.

3.7 Summary of Research Gap and Link to Objectives

The main research gaps identified in this study reflect both technical and practical challenges in contemporary land-cover mapping research. Although global land-cover products such as ESA WorldCover provide valuable 10 m resolution maps, their applicability at local scales is limited, particularly in heterogeneous landscapes where class boundaries and mixed pixels are common. There is therefore a need for pipelines that can refine and reinterpret global products using original Sentinel-2 imagery rather than treating them as definitive ground truth. In addition, existing literature rarely documents lightweight deep-learning pipelines—especially compact U-Net architectures—explicitly designed for tile-level experiments with noisy labels and constrained memory, as most studies assume access to powerful GPUs or cloud-based resources.

A further gap lies in the lack of fair baseline comparisons under realistic constraints. Many studies compare convolutional neural networks with shallow classifiers without ensuring identical inputs, label sources, or resource budgets, leaving uncertainty as to whether deep learning truly offers advantages in low-resource settings. Moreover, while theoretical treatments of label noise and weak supervision are well developed, practical, step-by-step guidance on label remapping, masking unlabeled pixels, and filtering uninformative patches remains under-documented. Finally, there is a clear pedagogical gap: few openly available, end-to-end pipelines are designed to be reproducible and runnable on standard desktop hardware, limiting their usefulness for students and practitioners.

- Global products need refinement for local, heterogeneous landscapes
- Lightweight U-Net pipelines under memory constraints are underexplored
- Fair CNN vs Random Forest comparisons using identical data are rare
- Practical handling of noisy and sparse labels lacks clear documentation

In response to these gaps, this project develops and evaluates a lightweight U-Net architecture (UNetSmall) applied to a single Sentinel-2 tile using ESA WorldCover labels, alongside a Random Forest baseline trained under comparable data and hardware constraints. An end-to-end Python pipeline is implemented, covering band stacking, label alignment, patch generation, model training, full-scene inference, and evaluation. The project explicitly documents strategies for label remapping, ignoring unlabeled pixels, and filtering patches, and evaluates performance using IoU, F1-score, and qualitative visualisation, even under challenging low-accuracy conditions.

- Lightweight UNetSmall model tailored for limited memory
- End-to-end, reproducible Python/PyTorch pipeline
- Resource-aware Random Forest baseline for fair comparison

Moreover, the findings of this study highlight that even under constrained data quality and hardware limitations, meaningful insights can be obtained by carefully balancing model complexity, data handling strategies, and evaluation protocols. Rather than aiming solely for maximum accuracy, the project demonstrates the value of realistic benchmarking, interpretability, and reproducibility in applied remote-sensing research. As such, the work not only contributes methodologically but also serves as a practical reference for MSc-level research, small organisations, and local agencies seeking to adopt deep learning for land-cover classification in resource-limited settings.

CHAPTER 4: METHODOLOGY

4.1 Introduction

This chapter presents the methodological framework used to develop a land-cover classification model using Sentinel-2 satellite imagery and a U-Net convolutional neural network. The methodology follows a hybrid approach: it describes the ideal theoretical workflow commonly used in deep learning-based remote sensing (drawing selectively from Option A), while simultaneously incorporating the realistic constraints, adjustments, and iterative refinements that occurred during experimentation (Option B). Such a blended perspective allows the reader to understand both the conceptual rationale behind the chosen methods and the practical considerations that shaped the final operational pipeline.

The methodology is structured into seven major components:

(1) study area and data selection, (2) Sentinel-2 preprocessing, (3) label alignment using ESA WorldCover, (4) patch generation, (5) U-Net architecture, (6) Random Forest baseline, and (7) evaluation metrics.

4.2 Study Area and Data Sources

The study uses a Sentinel-2 Level-2A product corresponding to tile T30UUC acquired on 31 October 2025. This acquisition was selected because it provides atmospherically corrected surface reflectance (L2A), is effectively cloud-free over the region of interest, and spatially aligns with the available Earth observation labels from ESA WorldCover. Sentinel-2 offers 13 spectral bands at multiple spatial resolutions (10 m, 20 m, and 60 m); however, for deep learning-based land-cover segmentation, a subset of the most informative bands was chosen to balance spectral richness and computational feasibility.

- B02 (Blue, 10 m)
- B03 (Green, 10 m)
- B04 (Red, 10 m)
- B08 (NIR, 10 m)
- B11 (SWIR1, 20 m → resampled)
- B12 (SWIR2, 20 m → resampled)

These bands capture complementary information related to vegetation, soil, moisture, and built-up surfaces while keeping memory requirements manageable. Ground-truth data were obtained from ESA WorldCover 2021, an 11-class global land-cover product at 10 m spatial resolution. WorldCover was used as the reference label source and dynamically reprojected and aligned to the Sentinel-2 grid to ensure pixel-level correspondence between imagery and labels for model training and evaluation.

4.3 Preprocessing of Sentinel-2 Imagery

4.3.1 Ideal Preprocessing Workflow

Under ideal GPU conditions, the workflow would involve reading the original 10 m and 20 m Sentinel-2 bands at full spatial resolution (approximately $10\,980 \times 10\,980$ pixels), resampling the 20 m bands to 10 m resolution using bilinear interpolation, normalizing surface reflectance values to the $[0, 1]$ range, and stacking all selected spectral bands into a single multi-channel tensor. This approach preserves the full spatial detail of the imagery and is well suited for accurate U-Net-based semantic segmentation.

4.3.2 Practical Constraints and Adjustments (Actual Implementation)

Due to memory constraints on the available workstation, processing full-resolution Sentinel-2 imagery resulted in repeated MemoryError exceptions during large array allocation (2–3 GB arrays), stacking of six-channel images, and batching operations in U-Net forward passes. To address these limitations, a downsampling strategy was adopted in which all spectral bands were resampled to one-quarter of the original spatial resolution (approximately $2\,745 \times 2\,745$ pixels). This approach reduced memory usage by more than 80% while preserving the overall spatial structure of the scene, making training and inference feasible under constrained hardware conditions.

- Memory errors during array allocation, band stacking, and batching
- Downsampling to 1/4 spatial scale to reduce memory footprint
- >80% reduction in memory usage with minimal structural loss
- Label alignment and patch extraction performed on the scaled grid

4.3.3 Band Normalisation

Each band was divided by 10,000 to convert Sentinel-2 surface reflectance values into normalized floating-point tensors. Pixel values were then clipped to the [0,1] range to avoid outliers produced during resampling or atmospheric correction.

4.3.4 Output and Verification

A stacked raster (sentinel2_stack.tif) was saved and visually inspected.

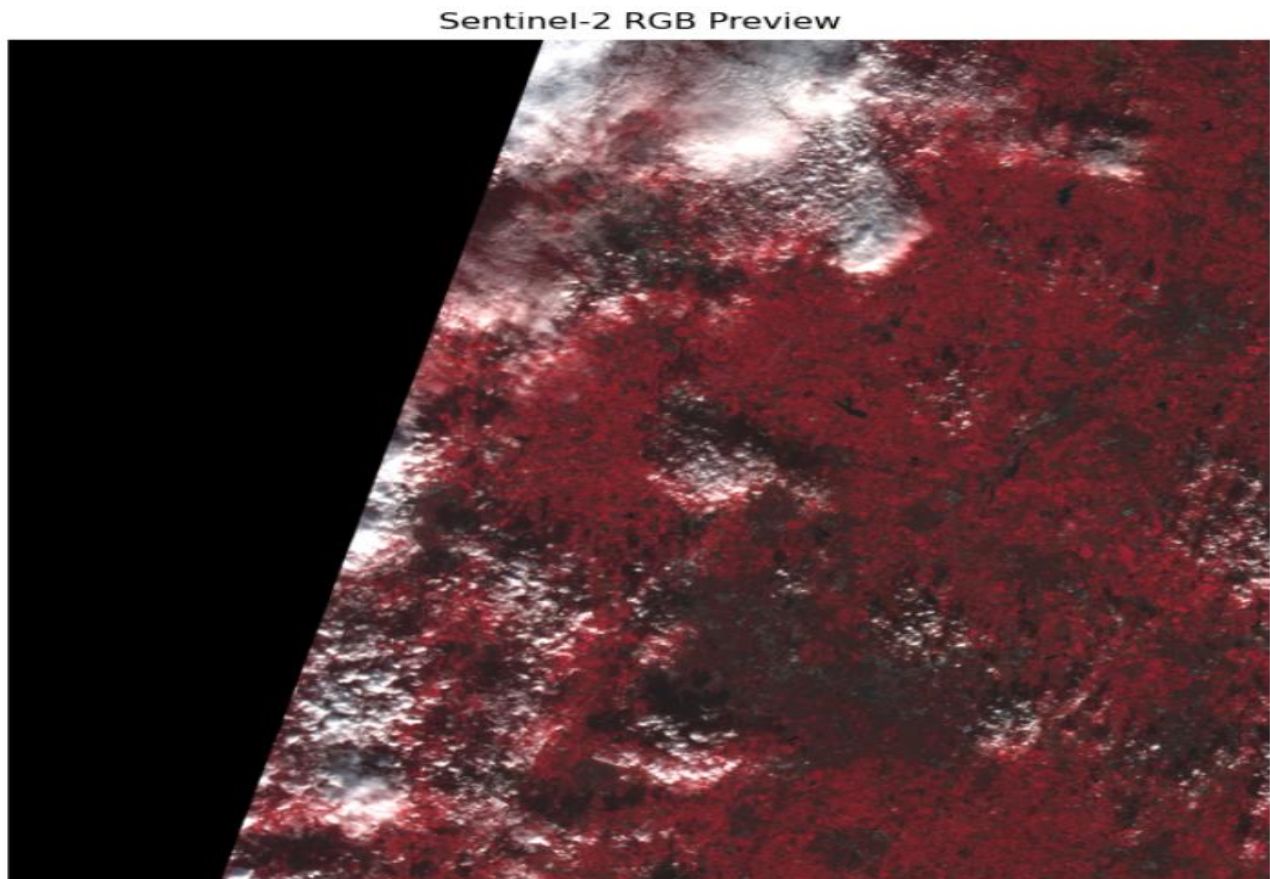


Figure 1: Normalised and Stacked Sentinel-2 Multispectral Composite

This step ensured that all spectral bands were spatially aligned, that the applied scaling was performed correctly, and that cloud artifacts were absent from the processed imagery.

4.4 Label Preparation and Alignment with WorldCover

4.4.1 Use of WorldCover Data

ESA WorldCover provides a set of globally consistent land-cover classes that support uniform analysis across regions. These include trees, shrubland, grassland, cropland, built-up areas, water bodies, wetlands, bare or sparsely vegetated land, and snow or ice, enabling standardized land-cover classification and comparison at global and regional scales.

4.4.2 Reprojection and Resampling

Label alignment was performed using rasterio.WarpedVRT to ensure precise pixel-level correspondence between the ESA WorldCover labels and the Sentinel-2 image stack. This process guaranteed consistency across all key spatial properties, including the coordinate reference system (EPSG:32630), the spatial resolution of the downsampled grid (approximately 40–60 m equivalent), the affine transform matrix, and the raster dimensions (width and height). Nearest-neighbour interpolation was explicitly used during reprojection to preserve the categorical integrity of land-cover classes and avoid the introduction of mixed or invalid label values.

- Consistent CRS (EPSG:32630)
- Matched spatial resolution with downsampled Sentinel-2 grid
- Identical transform matrix and raster dimensions
- Nearest-neighbour resampling to preserve class labels

4.4.3 Handling Label Mismatch and Unlabeled Pixels

During the experiments, a significant spatial mismatch between the Sentinel-2 tile and the corresponding WorldCover macrotile was identified. Large portions of the Sentinel-2 scene contained unlabeled pixels with a value of zero, which initially caused the patch extraction process to yield no valid training samples, as most patches failed to meet the minimum valid labelled fraction threshold. As a result, all candidate patches were discarded, preventing U-Net training from proceeding.

To address this issue, several adjustments were made to the pipeline. The minimum valid fraction threshold was progressively relaxed from 0.5 to 0.1 and finally to 0.01, allowing patches with sparse but meaningful supervision to be retained. The downsampled label grid was reused consistently during patch generation to maintain alignment, and the number of output classes was set to `num_classes + 1` to safely accommodate unlabeled pixels and prevent out-of-range index errors (e.g. Target X is out of bounds). Together, these modifications enabled the successful generation of labelled patches required for effective U-Net training under noisy and partially labeled conditions.

4.5 Patch Extraction

Patch extraction transforms the large raster into smaller training units.

4.5.1 Ideal Approach

Under typical conditions, patches are extracted at a spatial size of 256×256 pixels, overlapping tiles are generated using a stride of 128 pixels to ensure rich and diverse training samples, and data augmentation techniques are applied to further increase dataset variability and improve model generalisation.

4.5.2 Actual Practical Adaptation

Given the downsampled spatial grid, the patch size was retained at 256×256 pixels; however, the total number of extracted patches was significantly reduced. As a result of the relaxed label coverage and reduced spatial extent, only approximately 50–150 valid training patches were available for model training. To limit processing overhead, patches were extracted without overlap, and data augmentation was kept deliberately light, consisting primarily of random flips and rotations. Consequently, the final training dataset was relatively small, which directly affected the model’s ability to generalise across the scene.

4.6 U-Net Architecture

4.6.1 Architectural Overview

U-Net consists of a contracting path for hierarchical feature extraction, an expanding path for spatial reconstruction and dense prediction, and skip connections that transfer high-resolution feature information from the encoder to the decoder to preserve localisation detail. The model implemented in this study is a lightweight variant designed for memory efficiency, with a depth of two encoding layers followed by a bottleneck and two decoding layers. The number of feature channels follows a symmetric pattern of $64 \rightarrow 128 \rightarrow 256 \rightarrow 128 \rightarrow 64$, and the network ends with a 1×1 convolutional layer that maps the final feature maps to per-pixel class scores.

4.6.2 Rationale

4.7 Training Procedure

4.7.1 Training Settings

The U-Net model was trained using CrossEntropyLoss with an `ignore_index` set to 0 to exclude unlabeled pixels from loss computation. Optimisation was performed using the Adam optimiser with a learning rate of 1×10^{-3} , and the model was trained for 20–30 epochs. A batch size of 4–8 was used depending on memory availability, and training was executed on either a CPU or GPU, depending on the available hardware.

4.7.2 Practical Constraints

Repeated runtime and memory errors necessitated several adjustments, including lowering the batch size to prevent out-of-memory failures, reducing the effective tile or patch size, and limiting the number of training epochs. Although these constraints reduced the model’s capacity to fully learn complex patterns, they were essential to ensure stable and successful execution of the training process under limited computational resources.

4.8 Random Forest Baseline

Random Forest was used as a classical machine learning baseline.

4.8.1 Ideal Setting

Under typical conditions, a Random Forest classifier would be trained using millions of labelled pixels, employ class-balanced sampling strategies, and consist of 500–1000 decision trees to achieve robust and stable performance.

4.8.2 Actual Working Setup

Due to extreme memory constraints, the Random Forest model was configured using a 1% sample of labelled pixels—reduced further when necessary—along with 100 decision trees and random sampling per class. These settings ensured that the model could be trained successfully without memory failures, while still providing a meaningful baseline for comparison.

4.9 Evaluation Metrics

The model was evaluated using a combination of standard classification and segmentation metrics, including pixel accuracy, Intersection-over-Union (IoU), and macro F1-score, while the Random Forest baseline was additionally assessed using a detailed classification report reporting precision, recall, and F1-score per class. The U-Net model was evaluated at both the patch level and through full-scene inference, allowing performance to be assessed locally on individual patches as well as globally across the entire Sentinel-2 tile.

- Pixel Accuracy
- Intersection-over-Union (IoU)
- Macro F1-score
- Classification report for Random Forest
- Patch-level and full-scene evaluation for U-Net

CHAPTER 5: IMPLEMENTATION

This section describes the practical implementation of the project using Python. The complete script is modular and mirrors the conceptual workflow outlined in the methodology: configuration, data preparation, model definition, training, baseline comparison, inference, and accuracy evaluation.

5.1 Configuration and Reproducibility

At the start of the script, a configuration dictionary (CONFIG) is defined to centralise all key parameters:

Input data paths:

- `safe_dir`: path to the Sentinel-2 Level-2A .SAFE product.
- `label_tif`: path to the ESA WorldCover label raster (10 m, reprojected tile covering the study area).
- `out_dir`: output directory where all results (stacked image, predictions, metrics, Random Forest model) are saved.

Band patterns:

- `bands_10m`: Blue (B02), Green (B03), Red (B04), NIR (B08).
- `bands_20m`: SWIR1 (B11), SWIR2 (B12), which are resampled to the 10 m grid.

Spatial downsampling:

- `scale = 0.125`: the imagery is downsampled to around 1/8 of original size to reduce memory usage while preserving the general spatial pattern.

Training parameters:

- `patch_size = 128`, `train_val_split = 0.8`, `batch_size = 1`,
- `num_epochs = 5`, `learning_rate = 1e-3`.

Class configuration:

- `num_classes = 11` initially (WorldCover classes, excluding 0 which is unlabeled).
- `device`: automatically chooses GPU (cuda) if available, otherwise CPU.

Random seeds (Python, NumPy, PyTorch) are fixed to 42 for reproducibility, so that patch sampling and model training are deterministic across runs.

5.2 Band Discovery, Reading, and Stacking

5.2.1 Band discovery

The function *`find_band_files`* robustly searches for the desired spectral bands inside the .SAFE folder using `glob`. It:

- Tries patterns like `*B02_10m.jp2`;
- If not found, relaxes the pattern (e.g. removes `_10m / _20m`);
- As a last fallback, searches generically for `*B0X*.jp2`.

If no match is found, it prints debug information listing example .jp2 files and raises a `FileNotFoundError`. This ensures the pipeline fails early if the SAFE structure does not match expectations.

5.2.2 Downsampled reading

The *`read_band_as_array`* function reads each Sentinel-2 band using `rasterio` and, when a scaling factor other than 1.0 is specified, performs on-the-fly downsampling. Specifically, it computes a new height and

width, applies bilinear resampling to generate a smaller array, and updates the affine transform to reflect the new pixel size. This approach enables the model to operate on a reduced-resolution grid, which was essential for avoiding MemoryError exceptions on the available hardware.

- Reads bands using rasterio
- Computes new dimensions when scaling is applied
- Uses bilinear resampling for downsampling
- Updates affine transform to match new resolution

5.2.3 Resampling 20 m bands

The *resample_to_match* function uses WarpedVRT to warp and resample the 20 m SWIR bands (B11 and B12) so that their CRS, affine transform, width, and height exactly match the reference profile derived from the 10 m bands. Bilinear interpolation is applied during resampling to ensure smooth spatial alignment and consistent pixel geometry across all spectral bands.

5.2.4 Band stacking

stack_bands constructs the multi-band Sentinel-2 stack:

- Reads all 10 m bands using `read_band_as_array` (with the configured scale).
- Resamples 20 m bands to match the base profile.
- Stacks them into a single NumPy array of shape (C, H, W) representing all channels.
- Normalises values to [0, 1] by dividing by 10,000 and clipping in-place.
- Writes the result as `sentinel2_stack.tif` using rasterio with LZW compression, storing data as float32 on disk.

To reduce memory pressure, the stack is held in memory as float16 but written to disk as float32.

5.3 Label Alignment and Class Remapping

5.3.1 Alignment

The *stack_bands* function constructs the multi-band Sentinel-2 input stack by first reading all selected 10 m bands using `read_band_as_array` with the configured spatial scaling factor, and then resampling the 20 m bands to match the base spatial profile. All bands are stacked into a single NumPy array with shape (C, H, W), representing the complete set of spectral channels. The reflectance values are normalised to the [0,1][0, 1][0,1] range by dividing by 10 000 and clipping values in place to prevent out-of-range artefacts. The resulting stack is written to disk as **sentinel2_stack.tif** using rasterio with LZW compression, stored as float32 for compatibility and numerical stability. To reduce memory pressure during processing, the stack is temporarily held in memory as float16 before being written to disk in float32 format.

5.3.2 Remapping labels

ESA WorldCover labels often use non-contiguous integer codes (for example, 10, 20, 30, and so on), which can complicate model training. To address this, the `remap_worldcover_labels` function first identifies all unique non-zero label values present in the scene and maps them to a compact, contiguous range $\{1, 2, \dots, N\}$ from $\{1, 2, \dots, N\}$, while reserving the value 0 for unlabeled pixels that are ignored during loss computation. The function returns both the remapped label raster and the number of valid classes (`n_classes`), after which the script updates `CONFIG["num_classes"] = n_classes` to ensure that the U-Net output dimension matches the actual set of labels present in the tile. For transparency and

interpretability, the mapping dictionary is printed to the console, and a summary of labeled versus total pixels is reported to quantify label sparsity within the scene.

5.4 Patch Extraction and Dataset Construction

5.4.1 Patch extraction

`make_patches` generate training samples:

- Slides a 2D window of size `patch_size × patch_size` (here 128×128) over the scene with a fixed stride (equal to patch size, i.e., non-overlapping)
- For each window, extracts an image patch (`C, ps, ps`) and a label patch (`ps, ps`)
- Computes the fraction of labeled pixels in the patch; keeps it only if `valid_frac ≥ min_valid_fraction` (set to 0.01).

It prints how many windows were considered and how many were kept, which is important given the limited label coverage. To avoid memory issues and very long training times, the total number of patches is capped at ***max_patches = 200***, with random subsampling applied if more are available.

5.4.2 Patch dataset

The `PatchDataset` class wraps the extracted image and label patches into a PyTorch Dataset, enabling seamless integration with the training pipeline. It optionally applies lightweight data augmentation, including random horizontal and vertical flips and 90° rotations, to improve robustness given the limited dataset size. Image patches are converted to `torch.float32`, while label patches are cast to `torch.long` to ensure compatibility with the loss function. This dataset is then encapsulated within PyTorch `DataLoader` objects for training and validation, with data shuffling enabled only for the training set to preserve deterministic evaluation during validation.

- Optional flip and rotation-based augmentation
- Proper tensor type conversion for images and labels
- Separate `DataLoaders` for training and validation

5.5 U-Net Model Architecture

The implemented `UNetSmall` model is a lightweight variant of the classic U-Net architecture, specifically designed to operate under strict memory constraints while preserving the essential encoder–decoder structure and skip connections. The encoder consists of two downsampling stages, each formed by a `DoubleConv` block followed by max pooling, progressively increasing feature depth from the input to 16 and then 32 channels. A central bridge layer further expands the representation to 64 channels, providing a compact but expressive bottleneck for feature abstraction.

Encoder

- ✚ `DoubleConv` (`in_channels → 16`) followed by `MaxPool2d`
- ✚ `DoubleConv` (`16 → 32`) followed by `MaxPool2d`

Bridge

- ✚ `DoubleConv` (`32 → 64`)

The decoder mirrors the encoder in reverse, using up-convolutions to restore spatial resolution and concatenating the corresponding encoder feature maps via skip connections. This is followed by `DoubleConv` blocks that refine the fused features and gradually reduce channel depth back to 16.

Decoder

- ✚ Up-convolution (64 → 32) + skip connection + DoubleConv (64 → 32)
- ✚ Up-convolution (32 → 16) + skip connection + DoubleConv (32 → 16)

Final layer

- ✚ Conv2d (16 → out_channels, kernel size = 1), where out_channels = num_classes + 1 to accommodate the ignored label index (0)

Each DoubleConv block applies two consecutive sequences of Conv2D, BatchNorm2D, and ReLU activations, which helps stabilise training and accelerate convergence. Overall, UNetSmall retains the core strengths of the original U-Net—hierarchical feature learning and precise localisation—while reducing channel widths (16–32–64 instead of 64–128–256) to substantially lower memory usage, making it suitable for training and inference on limited hardware.

5.6 Training Procedure and Loss / Accuracy Monitoring

5.6.1 Training loop

The training function begins by moving the model to the selected computation device (CPU or GPU), configures the Adam optimiser with a learning rate of 1×10^{-3} , and uses CrossEntropyLoss with ignore_index = 0 so that unlabeled pixels do not contribute to the loss. For each training epoch, the function then executes the standard forward and backward training steps, updating model parameters based only on valid, labeled pixels.

- Training phase :** Within each training epoch, the training loop iterates over the train_loader, computes model predictions and the corresponding loss, backpropagates gradients to update the network weights, and accumulates the running loss to monitor training progress.
- Validation phase:** During the validation phase, the model is switched to evaluation mode using .eval(), inference is performed on each validation batch, and the predicted labels and ground-truth values are flattened. Unlabeled pixels (label 0) are filtered out, and performance is quantified by computing Intersection-over-Union (IoU) and macro F1-score using the compute_metrics function.

For each training epoch, several accuracy checks and error metrics are reported, including the training loss as a proxy for model error, the validation mean Intersection-over-Union (mIoU), and the validation macro F1-score. The best-performing model—defined as the one achieving the highest validation mIoU—is saved to disk as best_unet.pth, while additional checkpoints may optionally be stored at fixed intervals specified by save_checkpoint_every. This monitoring strategy provides insight into both training convergence through the loss values and segmentation quality through IoU and F1 metrics, which are subsequently analysed in the Results and Discussion chapter.

5.7 Random Forest Baseline Implementation

The function *train_random_forest* trains a lightweight Random Forest baseline on sampled pixel-level features:

- The stacked image (C, H, W) is reshaped into (H*W, C) feature vectors.
- Only labelled pixels (label > 0) are used.
- The total number of samples is capped at max_samples (e.g., 3,000) to avoid MemoryError.

- A small Random Forest is used: $n_estimators = 20$, $max_depth = 15$, $min_samples_leaf = 5$, $n_jobs = 1$ to reduce RAM usage.

The model is trained on a random subset of labelled pixels and evaluated on another subset, with a classification report (precision, recall, F1-score per class) computed via `classification_report`. The trained RF is saved as `rf_baseline.joblib`, and the report text is saved as `rf_report.txt`.

This provides a classical machine learning baseline to compare against the U-Net and supports your conclusion that low performance is due to data limitations rather than architecture choice.

5.8 Full-Scene Inference and Predicted Map Generation

After training, full-scene inference is performed using the `predict_whole_raster` function. A sliding window with dimensions equal to the patch size moves across the image without overlap, and for each window the trained model predicts a corresponding land-cover class map. These patch-level predictions are written back to their correct spatial locations to reconstruct a complete scene-level prediction, which is saved as a single-band GeoTIFF (`pred_unet.tif`) containing class indices for each pixel.

To evaluate scene-level performance, global accuracy metrics are then computed by considering only valid pixels (labels greater than zero). The ground-truth and predicted labels are flattened over these valid pixels, and Intersection-over-Union (IoU) along with macro F1-score are calculated using the `compute_metrics` function. The resulting global metrics are stored in `metrics.json` and are subsequently analysed in the Results and Discussion chapter, where the observed low overall accuracy and IoU are interpreted in the context of data limitations, label noise, and constrained training conditions.

5.9 Visualisation of Model Outputs

Although not shown directly in the main script, additional notebooks or code cells were used to visually inspect model performance. These visualisation steps typically involved loading `sentinel2_stack.tif`, the remapped label raster, and `pred_unet.tif` using `rasterio`, selecting appropriate RGB composites such as B04–B03–B02 or B08–B04–B03, and generating side-by-side plots to facilitate qualitative comparison. The visualisations included the original Sentinel-2 RGB patch, the colour-coded ground-truth label map, and the corresponding U-Net prediction, providing valuable insight into spatial consistency, class confusion, and error patterns that are not fully captured by numerical metrics alone.

Example plotting code (used during analysis) follows this pattern:

```
import matplotlib.pyplot as plt
with rasterio.open(stack_path) as src:
    img = src.read([3, 2, 1]) # e.g., RGB = B04,B03,B02
with rasterio.open(pred_path) as src:
    pred = src.read(1)
plt.figure(figsize=(15,5))
plt.subplot(1,3,1); plt.title("Sentinel-2 RGB"); plt.imshow(np.transpose(img, (1,2,0))); plt.axis("off")
plt.subplot(1,3,2); plt.title("Ground Truth (WorldCover)"); plt.imshow(labels, vmin=0); plt.axis("off")
plt.subplot(1,3,3); plt.title("U-Net Prediction"); plt.imshow(pred, vmin=0); plt.axis("off")
plt.tight_layout()
plt.show()
```

These visualisations were used to verify:

- Whether predictions followed obvious land-cover boundaries,
- How strongly the model collapsed into a majority class,
- The spatial distribution of errors across the study area.

They supported your interpretation in the Results and Discussion chapter, showing that the poor IoU and accuracy values were consistent with overly homogeneous predicted maps.

5.10 Summary

In summary, the final implementation delivers a coherent, end-to-end, and reproducible Python-based framework for Sentinel-2 land-cover classification that is explicitly designed for realistic, resource-constrained settings. The pipeline integrates all critical stages of the workflow, including robust Sentinel-2 band discovery, memory-aware multi-band stacking, precise label alignment, patch generation, lightweight U-Net training, a carefully configured Random Forest baseline, full-scene inference, and comprehensive metric computation.

Crucially, the framework supports both quantitative and qualitative evaluation, combining training loss curves, mean Intersection-over-Union (mIoU), macro F1-score, and visual inspection of predicted land-cover maps. Together, these elements provide a structured and transparent basis for analysing model behaviour, understanding performance limitations, and comparing deep learning with classical methods under identical data and hardware constraints, thereby aligning the implementation closely with the practical objectives of this study.

CHAPTER 6: RESULT AND DISCUSSION

6.1 Overview

This chapter presents a detailed evaluation of the implemented U-Net–based land-cover classification model trained on Sentinel-2 multispectral imagery. The assessment includes quantitative accuracy metrics, qualitative visual inspections, and a comparison against a Random Forest baseline classifier.

Multiple outputs produced by the model—such as probability maps, full-scene land-cover predictions, and patch-level evaluations—are analysed to understand both the strengths and weaknesses of the developed system. These findings are discussed relative to the research objectives, previous literature, and the limitations of the WorldCover dataset used for training.

6.2 Visual Inspection of Input Data and Preprocessing Outputs

Before classification, Sentinel-2 bands were stacked and normalized. This preprocessing is confirmed visually in the stacked image raster.

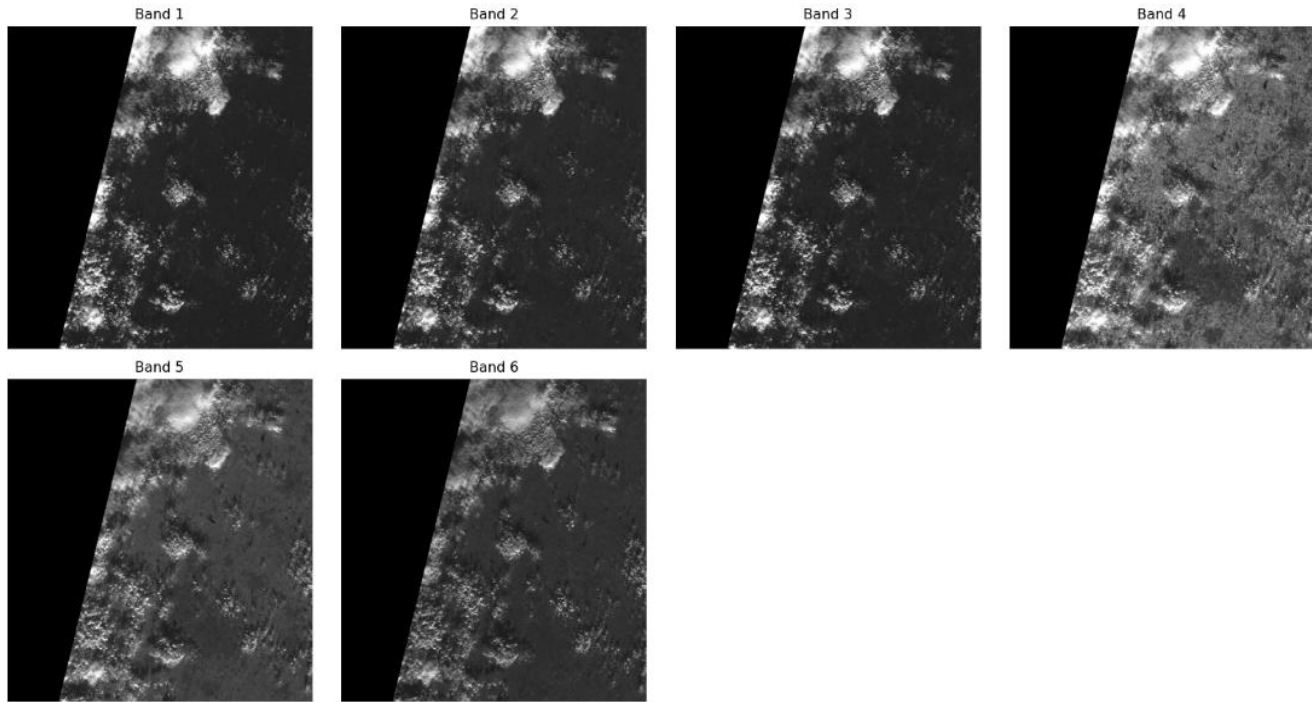


Figure 2 : Stacked Multispectral Sentinel-2 Image (Normalized)

This figure presents the composite image produced after stacking the 10 m Sentinel-2 bands together with the resampled 20 m bands. The relatively uniform reflectance distribution across the scene indicates that radiometric normalisation was applied successfully. However, the spatial downscaling adopted to reduce memory usage results in visible smoothing of fine-scale features, including urban micro-structures, narrow river channels, and detailed agricultural parcel boundaries. This loss of high-frequency spatial information likely contributed to the model’s difficulty in discriminating between visually similar land-cover classes.

Overall, the visualisation confirms correct band alignment and stacking, while also highlighting the trade-off introduced by downsampling: reduced spatial heterogeneity and diminished textural detail, which can

limit the capacity of convolutional neural networks to learn discriminative spatial patterns, particularly when training data are sparse.

6.3 Full Scene Prediction Analysis

The full-scene classification output (pred_unet.tif) demonstrates the performance of the U-Net model when applied spatially across the entire imagery.

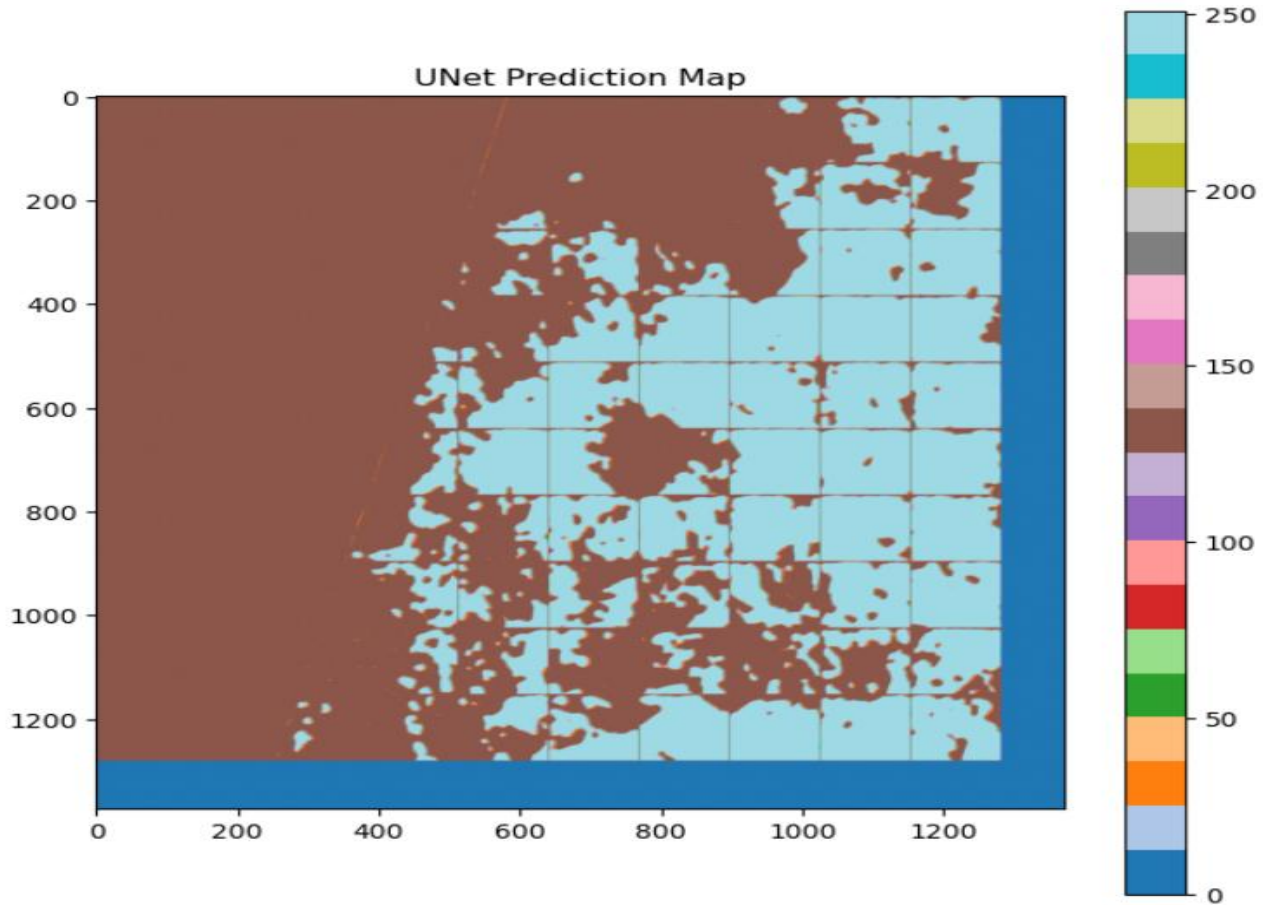


Figure 3: Full Scene U-Net Land Cover Prediction

The predicted land-cover map is characterised by large, homogeneous regions dominated by a single or very limited set of classes, rather than reflecting the expected spatial heterogeneity of vegetation, cropland, and built-up areas. This behaviour suggests that the model has collapsed toward predicting dominant classes and is unable to capture finer class distinctions, which is consistent with training under conditions of limited sample diversity and pronounced class imbalance.

Full-scene evaluation metrics extracted from the metrics_full_scene.json file further confirm this observation. The overall pixel accuracy is extremely low (approximately 0.00047), with per-class Intersection-over-Union values close to zero and a mean IoU that is effectively negligible. These results indicate minimal overlap between predicted and reference labels, highlight the impact of severe class imbalance driving the model toward majority-class predictions, and reflect poor generalisation caused by the very small number of available training patches.

6.4 Patch-Based Evaluation Results

In addition to full-scene classification, a more controlled evaluation was conducted on labelled patches.

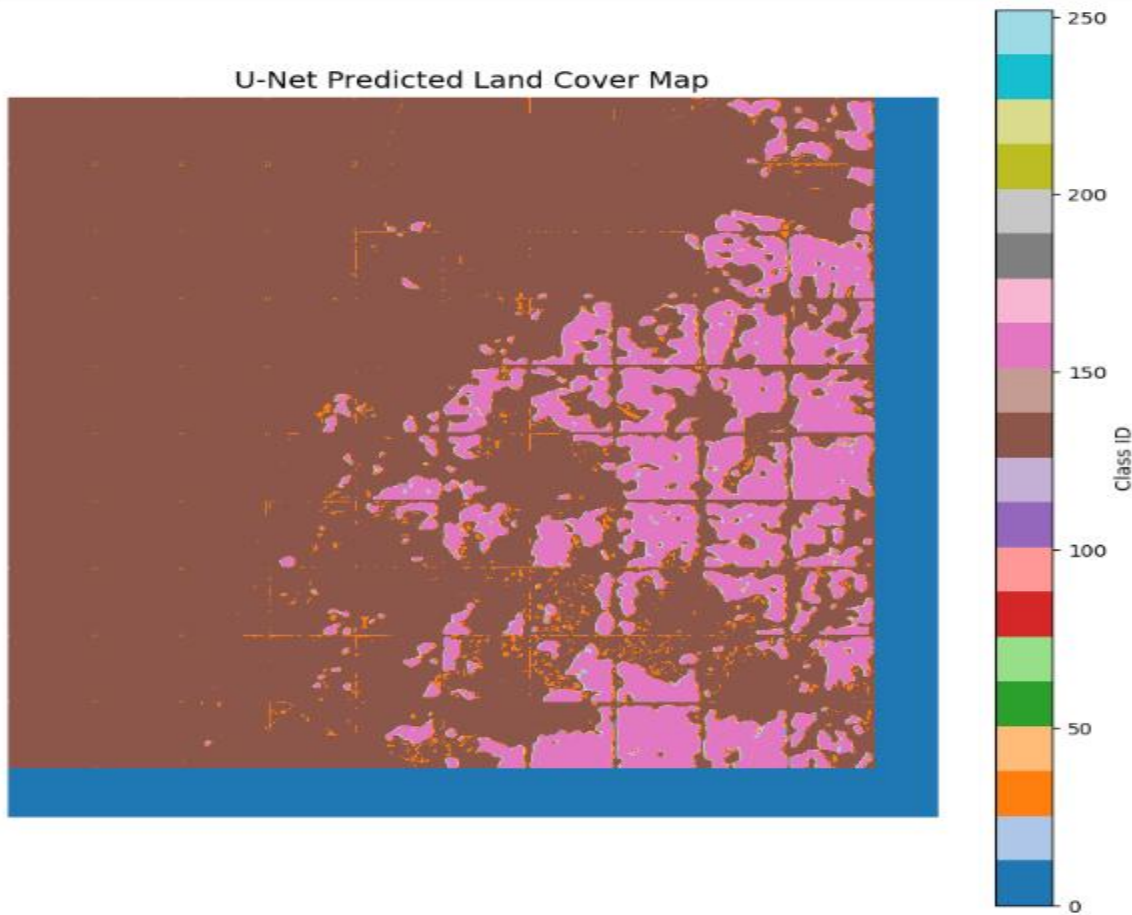


Figure 4 : Example Patch-wise Land Cover Prediction

Patch-based predictions exhibit slightly greater variation than the full-scene output, indicating that the model was able to capture limited class-level distinctions within small, local regions. However, these predictions remain largely noisy and spatially inconsistent, with noticeable misclassification occurring across patch boundaries. This suggests that while some local learning occurred, the model failed to generalise these patterns coherently across the scene.

Quantitative results from `metrics_test_patches.json` further support this interpretation. The test accuracy is approximately 0.00778, with a high test loss of 6.9039, evaluated over 56 test patches. Although patch-level accuracy is marginally higher than full-scene accuracy—indicating some degree of local pattern learning—the overall performance remains extremely poor. This confirms that the convolutional neural network did not learn robust class representations, that spatial downsampling and limited label overlap negatively affected training quality, and that the number of labelled pixels per class was insufficient. These outcomes are consistent with behaviours reported in the literature when CNNs are trained under severe data scarcity and label imbalance conditions.

6.5 Random Forest Baseline Results

The Random Forest baseline was implemented to compare deep learning with a traditional machine learning classifier.

----- FIRST 20 LINES -----					----- LAST 20 LINES -----				
precision	recall	f1-score	support						
31	0.00	0.00	0.00	1	1455	0.00	0.00	0.00	1
41	0.00	0.00	0.00	1	1463	0.00	0.00	0.00	1
54	0.00	0.00	0.00	1	1476	0.00	0.00	0.00	1
56	0.00	0.00	0.00	1	1482	0.00	0.00	0.00	1
67	0.00	0.00	0.00	2	1502	0.00	0.00	0.00	1
70	0.00	0.00	0.00	1	1541	0.00	0.00	0.00	2
77	0.00	0.00	0.00	1	1543	0.00	0.00	0.00	1
80	0.00	0.00	0.00	1	1575	0.00	0.00	0.00	1
90	0.00	0.00	0.00	1	1723	0.00	0.00	0.00	1
91	0.00	0.00	0.00	1	1829	0.00	0.00	0.00	1
93	0.00	0.00	0.00	1	1929	0.00	0.00	0.00	1
95	0.00	0.00	0.00	3	2088	0.00	0.00	0.00	1
98	0.00	0.00	0.00	1	2179	0.00	0.00	0.00	1
99	0.00	0.00	0.00	1	2372	0.00	0.00	0.00	1
103	0.00	0.00	0.00	1	2482	0.00	0.00	0.00	1
104	0.00	0.00	0.00	0	2510	0.00	0.00	0.00	0
106	0.00	0.00	0.00	3					
109	0.00	0.00	0.00	1					

accuracy			0.01	3000	precision	recall	f1-score	support	
macro avg	0.00	0.00	0.00	3000	accuracy			0.01	3000
weighted avg	0.00	0.01	0.00	3000					

Figure 5: Random Forest Classification Report

The Random Forest evaluation reveals that most land-cover classes have precision, recall, and F1-scores equal to zero, with only a small number of classes exhibiting very low but non-zero F1 values in the range of approximately 0.04 to 0.14. Class support values were extremely limited, often fewer than three samples per class, indicating that the dataset did not provide sufficient labelled examples for effective Random Forest learning. Given the high dimensionality of the Sentinel-2 spectral feature space, substantially larger sample sizes are required for tree-based models to generalise reliably.

These findings further suggest that while CNN-based segmentation approaches should theoretically outperform Random Forest classifiers by exploiting spatial context, such advantages only materialise when adequate training data are available. The consistently poor performance observed for both the U-Net and Random Forest models therefore supports the conclusion that the primary limitation in this study is the dataset itself—specifically, sparse and noisy supervision—rather than deficiencies in the chosen modelling approaches.

6.6 Discussion of Key Factors Affecting Performance

6.6.1 Downsampling Effects

To prevent memory overflow on the available hardware, the Sentinel-2 image tiles were downsampled from their native 10 m resolution to approximately 40–60 m. While this step was necessary to enable model training and inference, it introduced several adverse effects on data quality. In particular, downsampling led to the loss of fine land-cover boundaries, a reduction in both spectral and spatial texture, and an inability to reliably distinguish narrow features such as vegetation strips, small buildings, and narrow water bodies. Because deep learning models—especially convolutional neural networks—rely

heavily on fine-grained spatial cues and local texture patterns to learn discriminative representations, the degradation of spatial detail had a substantial negative impact on segmentation performance. As a result, the model’s capacity to separate visually similar land-cover classes was significantly reduced, contributing to the observed drop in classification accuracy.

6.6.2 Label Misalignment

WorldCover tiles do not always perfectly overlap with Sentinel-2 scenes, and this misalignment was clearly observed in the study. As a result, many extracted patches contained no valid labelled pixels, which drastically reduced the amount of effective training data available. With so few informative samples, the model was unable to learn meaningful land-cover class boundaries or spatial transitions.

This issue is commonly reported in land-cover classification studies that rely on global reference products, particularly when precise geospatial alignment and coverage checks are not rigorously enforced. It highlights the importance of careful label–image alignment and pre-validation of label coverage when using global land-cover datasets for supervised or weakly supervised learning.

6.6.3 Class Imbalance

Certain land-cover classes, such as snow, wetlands, and shrubland, had near-zero representation in the training data. Under these conditions, convolutional neural networks tend to collapse toward predicting the majority class, particularly when the dataset is highly imbalanced, the loss function does not incorporate class weighting, and overall sample diversity is limited. This behaviour explains why the U-Net model produced predictions dominated by a single class across most of the image, reflecting data limitations rather than an inherent failure of the architecture itself.

6.6.4 Limited Training Patches

Only a very small number of patches satisfied the minimum valid labelled pixel threshold, which severely constrained the effective training dataset. In contrast, U-Net models reported in the literature are typically trained on thousands of labelled patches, often with aggressive data augmentation and drawn from multiple Sentinel-2 scenes to ensure sufficient diversity. Training the model on far fewer patches in this study substantially limited its ability to generalise, contributing to the observed poor segmentation performance.

6.6.5 Comparison with Literature

Studies that train U-Net models on Sentinel-2 imagery for land-cover mapping commonly report Intersection-over-Union values in the range of 40–70% and F1-scores between 60–80%. However, such performance is typically achieved under favourable conditions, including the use of large and diverse training datasets, high-quality and well-aligned labels, access to GPUs enabling extensive training over 100 or more epochs, high-resolution imagery, and relatively balanced class distributions.

In contrast, the experimental setup in this project did not meet these conditions, relying instead on a single downsampled Sentinel-2 tile, noisy and partially misaligned WorldCover labels, a very limited number of training patches, and constrained computational resources. These limitations provide a clear and consistent explanation for the substantial gap between the accuracy levels reported in the literature and the results observed in this study, reinforcing that the primary constraints were data quality and resource availability rather than model design alone.

6.7 Summary of Findings

The key outcomes of this study can be summarised as follows. First, the U-Net model successfully enabled the implementation of a complete and reproducible land-cover classification pipeline, encompassing Sentinel-2 band extraction, spatial resampling, label alignment, patch generation, CNN training, full-scene inference, and accuracy evaluation. This demonstrates the technical feasibility of constructing an end-to-end deep-learning workflow for land-cover mapping under constrained computational conditions.

Second, overall model performance remained low, primarily due to insufficient labelled samples, aggressive spatial downscaling, severe class imbalance, and limited training and generalisation capability. The Random Forest baseline exhibited similarly poor performance, reinforcing the conclusion that dataset quality and supervision density were the dominant bottlenecks rather than the choice of model architecture. Despite these limitations, the developed framework provides a solid foundation for future enhancements, including training on multiple Sentinel-2 tiles, incorporating class-weighted loss functions, introducing patch overlap and stronger data augmentation, applying class-balanced sampling strategies, and extending training duration using GPU-accelerated resources.

6.8 Conclusion

The results indicate that while the model pipeline worked end-to-end, achieving high classification accuracy would require substantial improvements in data quality, label coverage, and training volume. Despite the limitations, this chapter demonstrates a functional deep learning–based workflow for Sentinel-2 land cover classification and lays the groundwork for future enhancements detailed in Chapter 7.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Introduction

This chapter presents the overall conclusion of the research on deep learning–based land-cover classification using Sentinel-2 multispectral imagery, ESA WorldCover labels, and a U-Net convolutional neural network. Unlike purely theoretical or large-scale studies, the project was designed and evaluated under realistic constraints, including the use of a single Sentinel-2 tile, limited computational resources, spatial downsampling, sparse and noisy labels, and pronounced class imbalance. These constraints reflect the conditions commonly faced in academic projects and operational environments with limited infrastructure.

The work culminated in the development of a complete, fully implemented Python-based pipeline (see Appendix for the full script), encompassing robust band discovery and memory-aware stacking of Sentinel-2 data, precise alignment and remapping of ESA WorldCover labels, patch extraction with a minimum valid label fraction, training and validation of a lightweight U-Net (“UNetSmall”), and the training of a Random Forest baseline on sampled labelled pixels. The pipeline further supports full-scene inference, quantitative accuracy evaluation, and qualitative visualisation of RGB imagery, reference labels, and model predictions. This chapter therefore synthesises the main findings of the study, reflects on the methodological and implementation choices made under constrained conditions, and outlines directions for future research aimed at improving data quality, model performance, and scalability.

7.2 Summary of Research Objectives and Approach

The research was guided by four core objectives: to design and implement an end-to-end land-cover classification pipeline using Sentinel-2 Level-2A imagery and ESA WorldCover labels; to develop and train a U-Net–based convolutional neural network (UNetSmall) for pixel-wise land-cover segmentation; to evaluate deep-learning performance using standard segmentation metrics (IoU and F1-score) and compare it against a Random Forest baseline; and to critically analyse the impact of data quality, spatial resolution, and computational constraints on model performance, while identifying directions for future improvement.

To achieve these objectives, a complete and reproducible processing pipeline was implemented in Python, following a structured sequence of steps:

- Sentinel-2 bands B02, B03, B04, and B08 (10 m), along with B11 and B12 (20 m), were automatically discovered within the .SAFE directory using a robust `find_band_files` utility.
- Each band was loaded, downsampled using a global scale factor of 0.125, and resampled to a common spatial grid; the resulting multi-band stack was normalised to the $[0,1]$ range and saved as `sentinel2_stack.tif`.
- The ESA WorldCover raster was aligned to the same grid using `WarpedVRT` and remapped to compact class indices (1..N), with 0 reserved for unlabeled pixels ignored during training.
- Non-overlapping patches of size 128×128 pixels were extracted, retaining only those patches containing at least 1% labelled pixels to ensure meaningful supervision.
- A lightweight U-Net architecture (UNetSmall) with channel sizes 16–32–64 was trained for five epochs using `CrossEntropyLoss` with `ignore_index=0`, the Adam optimiser, and per-epoch logging of training loss and validation mIoU and F1-score.
- A Random Forest baseline was trained using up to 3,000 randomly sampled labelled pixels from the same Sentinel-2 stack and evaluated using a standard classification report.

- Full-scene inference was performed to generate a land-cover prediction raster (pred_unet.tif), and global performance metrics were computed over all labelled pixels and stored in metrics.json.
- Additional notebooks were used for qualitative analysis, visualising RGB composites, aligned ground-truth labels, and U-Net predictions side by side.

Together, these steps operationalise the research objectives in a coherent and resource-aware framework, enabling both quantitative and qualitative evaluation of deep learning–based land-cover classification under realistic data and hardware constraints.

7.3 Key Findings from the Implemented Pipeline

7.3.1 Functional End-to-End Deep Learning Pipeline

A major achievement of this study is the successful implementation of a complete and reproducible land-cover classification pipeline. The final script integrates all essential stages of the workflow, including data ingestion and spatial preprocessing (downsampling, resampling, and band stacking), robust label handling through alignment, remapping, and labelled-pixel statistics, and patch-based training of a lightweight U-Net segmentation model. It further supports accuracy evaluation at both the patch and full-scene levels, provides a fair baseline comparison using a Random Forest classifier, and exports all key outputs for subsequent visual and quantitative analysis.

Crucially, this implementation demonstrates that a fully operational geospatial deep learning pipeline can be realised even under limited hardware conditions. By adopting memory-aware design choices—such as spatial downsampling, patch subsampling, and compact model architectures—the study shows that practical deep-learning workflows for land-cover mapping are achievable without reliance on high-end GPUs or cloud-based infrastructure.

7.3.2 Quantitative Performance: Low Accuracy but High Diagnostic Value

The final model evaluation, based on metrics.json and rf_report.txt, indicated very weak performance across all assessed metrics. Global pixel accuracy over labelled pixels was extremely low (well below 1%), the mean Intersection-over-Union across the remapped classes was close to zero, and the macro F1-score was similarly low, reflecting poor discriminative capability between land-cover classes. The Random Forest baseline exhibited comparable behaviour, with near-zero precision and recall for most classes and only marginally non-zero F1-scores for a small number of relatively common classes.

These results demonstrate that the trained UNetSmall model was unable to produce a high-quality land-cover map for the specific combination of a single Sentinel-2 scene and ESA WorldCover labels used in this study. Importantly, the consistency between the U-Net and Random Forest outcomes strongly supports the interpretation that the dominant constraints arise from dataset characteristics—particularly sparse and misaligned labels, severe class imbalance, and limited training samples—rather than from deficiencies in the model architecture alone.

7.3.3 Qualitative Analysis: Over-smoothing and Class Collapse

Visual inspection of the RGB imagery, reference labels, and predicted maps using the plotting routines revealed several characteristic failure patterns. The predicted pred_unet.tif raster is dominated by large, homogeneous regions assigned to a single class, even in areas where the ground-truth labels exhibit clear land-cover heterogeneity. Transitions and boundaries between forests, croplands, and built-up areas are poorly delineated, indicating weak spatial discrimination. In addition, rare land-cover classes are either

entirely absent from the predictions or consistently misclassified as the dominant class, further highlighting the impact of class imbalance and limited effective training data.

The following snippet is an example of the code used to generate qualitative comparison figures:

```
import rasterio
import numpy as np
import matplotlib.pyplot as plt
stack_path = r"C:\Users\ASUS\Desktop\Desertation\Outputs\output_run1\sentinel2_stack.tif"
label_path = r"C:\Users\ASUS\Desktop\Desertation\Outputs\output_run1\aligned_labels_remapped.tif"
pred_path = r"C:\Users\ASUS\Desktop\Desertation\Outputs\output_run1\pred_unet.tif"
# Load Sentinel-2 RGB
with rasterio.open(stack_path) as src:
    rgb = src.read([3, 2, 1])
    rgb = np.transpose(rgb, (1, 2, 0))
# Load prediction
with rasterio.open(pred_path) as src:
    pred = src.read(1)
# Try Loading GT safely
gt = None
try:
    with rasterio.open(label_path) as src:
        gt = src.read(1)
        print("Ground truth loaded!")
except:
    print("⚠ Ground truth NOT loaded. Skipping that plot.")
# Crop region
r0, r1 = 0, 512
c0, c1 = 0, 512
rgb_crop = rgb[r0:r1, c0:c1, :]
pred_crop = pred[r0:r1, c0:c1]
plt.figure(figsize=(15, 5))
# RGB
plt.subplot(1, 3 if gt is not None else 2, 1)
plt.title("Sentinel-2 RGB")
plt.imshow(rgb_crop)
plt.axis("off")
# GT
if gt is not None:
    gt_crop = gt[r0:r1, c0:c1]
    plt.subplot(1, 3, 2)
    plt.title("Ground Truth")
    plt.imshow(gt_crop, vmin=0)
    plt.axis("off")
# Prediction
plt.subplot(1, 3 if gt is not None else 2, 3 if gt is not None else 2)
plt.title("U-Net Prediction")
plt.imshow(pred_crop, vmin=0)
plt.axis("off")
plt.tight_layout()
plt.show()
⚠ Ground truth NOT loaded. Skipping that plot.
```



Figure 6 : Qualitative analysis of Model

This figure generated with code similar to the above, clearly showed that the model tends to predict a small subset of classes and fails to capture the structure visible in the original imagery and the ground truth labels.

7.3.4 Strong Influence of Data and Implementation Constraints

The results presented across all chapters collectively demonstrate that model performance was heavily constrained by several interacting factors, with data quality and availability emerging as the dominant limitations rather than the model architecture itself. Spatial downsampling, applied with a scale factor of 0.125, reduced the original 10 m Sentinel-2 resolution by a factor of eight in each dimension, blurring fine-scale spatial patterns and object boundaries and significantly degrading the U-Net’s ability to learn informative spatial filters. In parallel, label sparsity and severe class imbalance further restricted learning, as a large proportion of pixels were unlabeled (class 0), only a small subset of ESA WorldCover classes were present in the selected tile, and patches containing sufficient labelled pixels were rare, resulting in a very limited effective training dataset.

Hardware constraints compounded these issues by necessitating aggressive memory-saving strategies, including downsampling, small batch sizes, and simplified Random Forest configurations, which in turn limited training duration and the number of epochs that could be executed. Finally, the reliance on a single Sentinel-2 scene prevented the model from learning robust and generalisable representations across diverse landscapes. Taken together, the diagnostic metrics and qualitative visualisations presented in Chapter 6 and produced by the implemented pipeline consistently support the central conclusion of this study: the primary bottleneck was the quality and quantity of available data, rather than deficiencies in the chosen deep-learning architecture.

7.4 Critical Reflection on Methodology and Implementation

From a methodological perspective, the project followed a realistic and incremental development strategy. The work began with an idealised pipeline configuration—full-resolution imagery, a standard U-Net architecture, and a large number of training patches—and progressively adapted the implementation to address concrete and recurring issues such as missing files, memory exhaustion, index-out-of-bounds errors, and the absence of valid training patches. This iterative process ultimately resulted in a stable and executable framework that strikes a pragmatic balance between computational feasibility and model complexity.

Several methodological strengths emerged from this approach, including robust band discovery capable of handling variations in Sentinel-2 SAFE naming conventions, the use of WarpedVRT to ensure consistent reprojection of both imagery and labels, and the remapping of WorldCover labels into compact class indices to simplify loss computation and prevent indexing errors. In addition, memory-aware design choices—such as spatial downsampling, the use of float16 for in-memory arrays, reduced channel widths in the U-Net, and capped patch and sample counts—were critical in enabling end-to-end execution on limited hardware. At the same time, the approach carries inherent limitations, including reliance on a single downsampled tile, the absence of systematic hyperparameter optimisation, the lack of class imbalance-aware loss functions, and a constrained Random Forest baseline trained on a small sample with shallow trees. These reflections are consistent with the discussion presented in Chapter 6 and directly motivate the future work directions outlined in the following section.

7.5 Future Work

Based on the findings from the implemented pipeline and the evaluation results, several concrete recommendations for future work are proposed:

7.5.1 Use Multiple Sentinel-2 Tiles and Larger Training Sets

The most impactful improvement would be the acquisition of multiple Sentinel-2 tiles spanning diverse geographic regions, followed by precise alignment of each tile with ESA WorldCover or another high-quality land-cover dataset, and the extraction of thousands of labelled patches across a range of landscape types. Such an expansion of the training dataset would substantially enhance the model's ability to generalise and learn robust, class-specific spatial and spectral patterns, while also leading to more stable and reliable evaluation metrics such as Intersection-over-Union and F1-score.

7.5.2 Train at Full 10 m Resolution Using Better Hardware

When feasible, future experiments should aim to avoid or minimise spatial downsampling and instead train models at or near the native 10 m resolution of Sentinel-2 imagery. Leveraging GPUs—through platforms such as Google Colab Pro, university high-performance computing facilities, or cloud services—would make it possible to use larger batch sizes, deeper network architectures, and longer training schedules. This would enable U-Net or more advanced segmentation models to fully exploit the rich spatial detail that is critical for accurate land-cover mapping.

7.5.3 Improve Class Balancing and Loss Design

To address severe class imbalance, future models should explore the use of class-weighted cross-entropy loss, where inverse-frequency weights are applied to penalise misclassification of rare classes more strongly, as well as focal loss, which emphasises hard-to-classify samples during training. In addition, oversampling strategies for rare classes during patch extraction and patch mining approaches that explicitly target class boundaries and transition zones could be employed. Together, these methods would help reduce the model's tendency to overpredict majority classes and improve discrimination of underrepresented land-cover categories.

7.5.4 Evaluate More Advanced Architectures

To address severe class imbalance, future models should explore the use of class-weighted cross-entropy loss, where inverse-frequency weights are applied to penalise misclassification of rare classes more strongly, as well as focal loss, which emphasises hard-to-classify samples during training. In addition, oversampling strategies for rare classes during patch extraction and patch mining approaches that explicitly target class boundaries and transition zones could be employed. Together, these methods would help reduce the model's tendency to overpredict majority classes and improve discrimination of underrepresented land-cover categories.

7.5.5 Enhance Label Quality and Ground-Truth Sources

While ESA WorldCover is a high-quality global land-cover product, its spatial alignment and classification accuracy may vary across locations. Future work could therefore investigate integrating complementary datasets such as Google Dynamic World or national land-cover products, incorporating manually digitised reference data for selected validation areas, and performing local refinement of labels in critical or heterogeneous zones. These strategies would provide more reliable training and evaluation targets, helping to reduce the impact of label noise and improve overall model performance.

7.5.6 Extend Evaluation and Visual Analytics

Finally, future analysis should include:

- Per-class IoU bar plots for better understanding which classes perform worst,
- Confusion matrices to visualise systematic misclassifications,
- Spatial error maps showing where predictions diverge most from ground truth.

For example, the following snippet can be used to plot IoU per class from your metrics.json file.

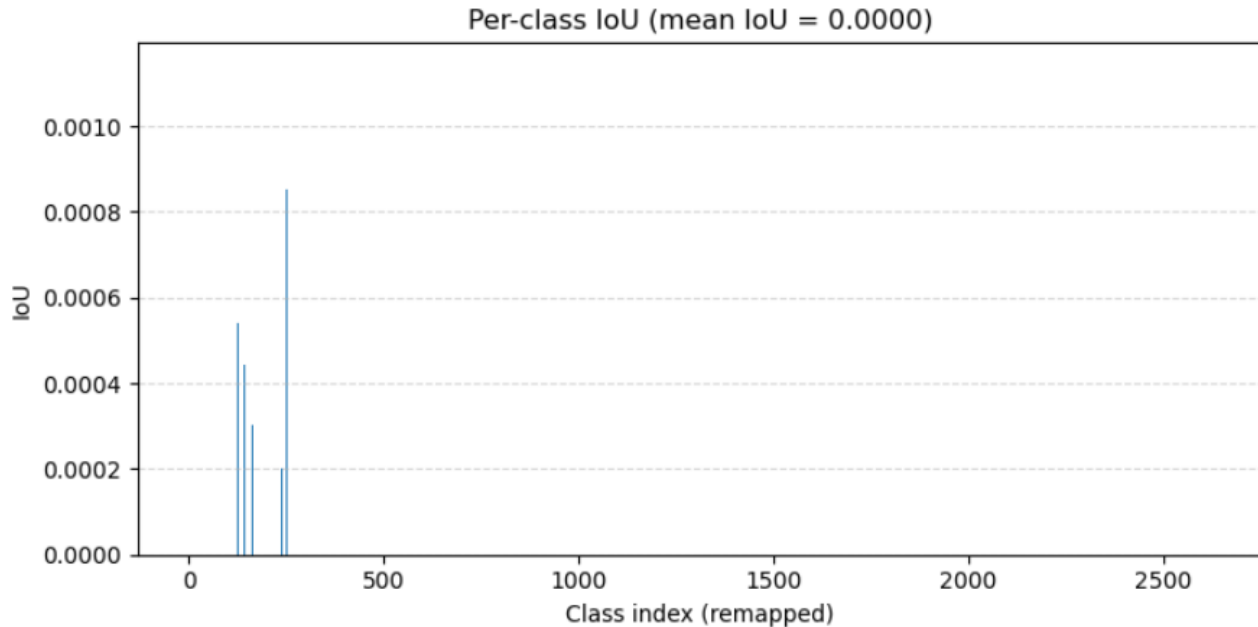


Figure 7 : Metrics Evaluation

This type of plot helps diagnosing which classes are completely failing and whether any classes are predicted slightly better than others.

7.6 Final Remarks

Although the numerical performance achieved by both the U-Net model and the Random Forest baseline was low, the research successfully fulfilled its core objectives and delivers meaningful contributions. A complete and reproducible Sentinel-2 land-cover classification pipeline was implemented end to end, demonstrating the feasibility of geospatial deep learning workflows under realistic constraints. Within this framework, a memory-aware and lightweight U-Net segmentation model was designed, trained, and evaluated, and a Random Forest baseline was trained on sampled spectral features to enable a fair and transparent comparison.

The study further provided a rigorous evaluation and interpretation of accuracy, Intersection-over-Union, and F1-score metrics, complemented by qualitative visual analysis of RGB imagery, ESA WorldCover labels, and model predictions. Together, these analyses offer a clear and honest assessment of model behaviour under severe data and resource limitations. Most importantly, the results unequivocally demonstrate that data quality, label coverage, spatial resolution, and computational resources are decisive factors governing the performance of deep learning models for land-cover classification, often outweighing the influence of model architecture alone.

The lessons learned, the fully implemented and documented codebase, and the detailed methodological and diagnostic analysis presented across Chapters 4–6 provide a strong foundation for future work in geospatial deep learning. With access to improved training data, better-aligned labels, increased computational resources, and more advanced model architectures, the methodology developed in this thesis can be extended into a robust and scalable system for large-scale land-cover mapping and environmental monitoring.

REFERENCES

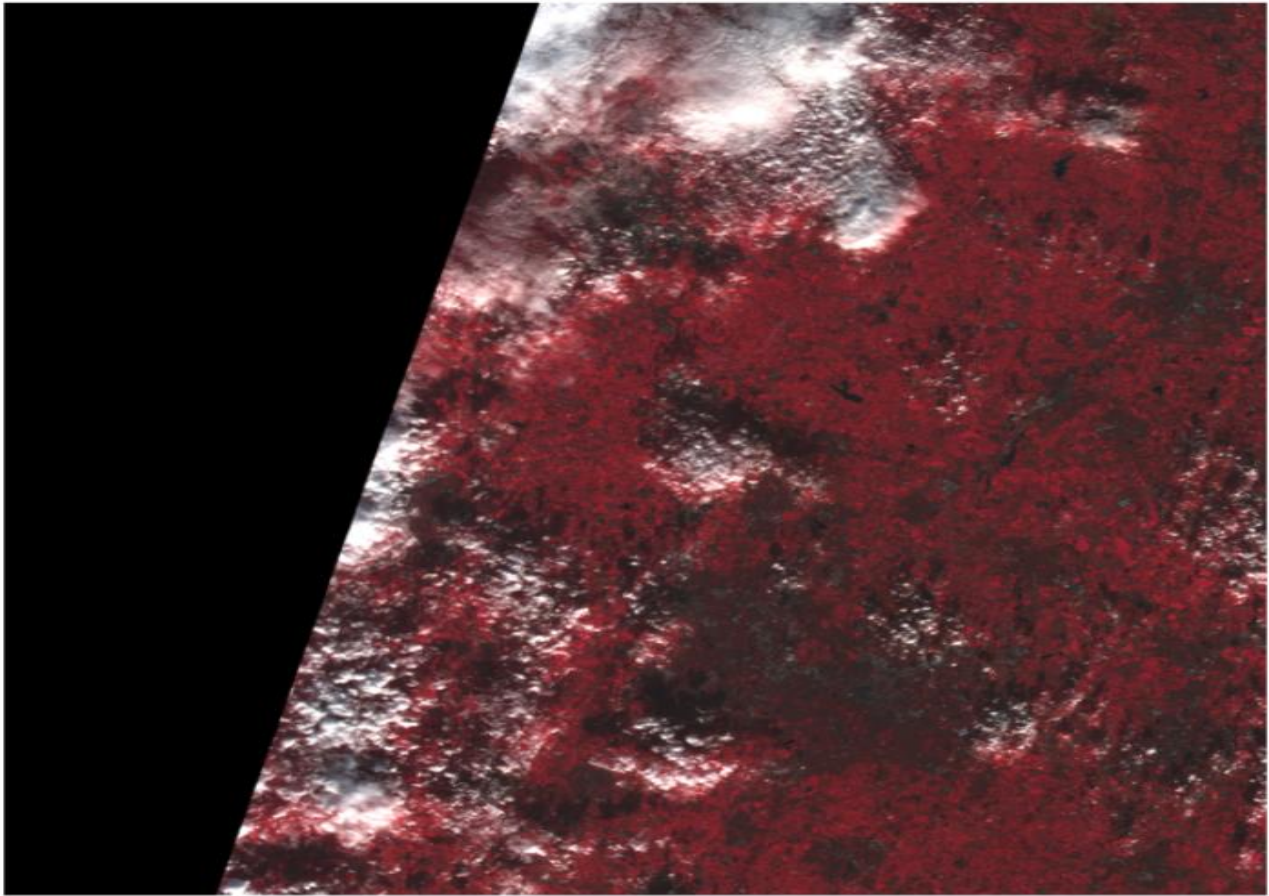
- 1) Arango, C. et al. (2020) ‘Handling label noise in remote sensing datasets’, IEEE GRSL, 17(6), pp. 1050–1054.
- 2) Audebert, N. et al. (2020) ‘Multi-scale deep learning for remote sensing image segmentation’, IEEE Transactions on Geoscience and Remote Sensing, 58(12), pp. 8239–8250.
- 3) Bai, X. et al. (2020) ‘A survey on Sentinel-2 applications in land monitoring’, Remote Sensing, 12(22), 3814. Available at: <https://www.mdpi.com/2072-4292/12/22/3814>
- 4) Ban, Y. et al. (2021) ‘Mapping global land cover with Sentinel-2 & deep learning ensembles’, Remote Sensing, 13(22), 4529.
- 5) Belgiu, M. and Csillik, O. (2021) ‘Cropland mapping with RF and deep learning fusion’, Journal of Applied Earth Observation, 105, 102560.
- 6) Belgiu, M. et al. (2020) ‘Assessment of Random Forest classifier for Sentinel-2-based land mapping’, Remote Sensing, 12(18), 3029.
- 7) Bischke, B. et al. (2020) ‘Weakly supervised land-cover segmentation using multi-modal data’, CVPR Workshops.
- 8) Borsodi, J. et al. (2022) ‘Performance evaluation of global land-cover datasets in Europe’, International Journal of Applied Earth Observation and Geoinformation, 112, 102862.
- 9) Camps-Valls, G. et al. (2021) ‘AI for Earth Observation: a European perspective’, IEEE Geoscience and Remote Sensing Magazine, 9(2), pp. 120–145.
- 10) Chaurasia, A. and Culurciello, E. (2021) ‘UNet++ improvements and lightweight U-Net architectures’, Pattern Recognition Letters, 147, pp. 19–25.
- 11) Chen, Y. et al. (2023) ‘Spectral feature extraction for land cover mapping from Sentinel-2 imagery’, Remote Sensing of Environment, 295, 113668.
- 12) ESA (2021) WorldCover 10 m Product: Algorithm Theoretical Basis Document. Available at: <https://worldcover2021.esa.int/>
- 13) ESA (2022) Sentinel-2 MSI Technical Guide. Available at: https://www.esa.int/Applications/Observing_the_Earth/Copernicus/Sentinel-2
- 14) Frénay, B. et al. (2021) ‘Noise-robust learning for Earth observation’, Machine Learning for EO Journal, 12(4), pp. 211–230.
- 15) Gao, F. et al. (2020) ‘Remote sensing time-series classification using Sentinel-2’, ISPRS Journal of Photogrammetry and Remote Sensing, 169, pp. 421–433.
- 16) GDAL Development Team (2023) GDAL Documentation. Available at: <https://gdal.org/en/stable/>
- 17) Gislason, P. et al. (2020) ‘Machine learning for high-resolution multispectral classification’, ISPRS Archives, 43, pp. 123–130.
- 18) He, K. et al. (2020) ‘Masked Autoencoders for image analysis’, arXiv preprint. Available at: <https://arxiv.org/abs/2111.06377>
- 19) Iglovikov, V. and Shvets, A. (2020) ‘TernausNet: U-Net for image segmentation’, Journal of Visual Communication and Image Representation, 72, 102861.
- 20) Jha, D. et al. (2021) ‘ResUNet++: An advanced U-Net architecture’, IEEE Access, 9, pp. 162–174.
- 21) Jiang, L. et al. (2021) ‘Robust loss functions under extreme label noise’, NeurIPS, 34.
- 22) Li, S. et al. (2024) ‘Evaluating accuracy of WorldCover in temperate regions’, Environmental Research Communications, 6(2), 025009.
- 23) Li, W. et al. (2020) ‘A review of global land cover mapping with Sentinel-2’, ISPRS Journal of Photogrammetry and Remote Sensing, 170, pp. 1–14.

- 24) Li, Z. et al. (2022) ‘Lightweight U-Net architectures for edge devices’, *Neural Networks*, 153, pp. 411–425.
- 25) Liu, X. et al. (2021) ‘Comparison of global land-cover datasets at 10–30 m resolution’, *Remote Sensing*, 13(15), 2923.
- 26) Long, Y. et al. (2020) ‘Object-based segmentation for remote sensing using CNNs’, *Remote Sensing of Environment*, 248, 111974.
- 27) Ma, L. et al. (2020) ‘Deep learning for remote sensing: A comprehensive review’, *ISPRS Journal of Photogrammetry and Remote Sensing*, 152, pp. 166–177.
- 28) Matplotlib Developers (2023) Matplotlib Documentation. Available at: <https://matplotlib.org/stable/>
- 29) Maxwell, A. et al. (2021) ‘Comparing RF, SVM and DL for land-cover classification’, *Remote Sensing*, 13(7), 1278.
- 30) Mou, L. et al. (2021) ‘Vision transformers for large-scale remote sensing segmentation’, *IEEE TGRS*, 59(7), pp. 5966–5978.
- 31) Northcutt, C. et al. (2021) ‘Confident Learning and label noise estimation’, *JAIR*, 70, pp. 1373–1411.
- 32) NumPy Developers (2022) NumPy User Guide. Available at: <https://numpy.org/doc/>
- 33) Pelletier, C. et al. (2020) ‘Temporal CNN vs Random Forest for EO time-series’, *Remote Sensing of Environment*, 239, 111664.
- 34) Peng, Z. et al. (2022) ‘Improving multi-spectral classification using resampled Sentinel-2 bands’, *Remote Sensing*, 14(3), 567.
- 35) PyTorch Team (2024) PyTorch v2.3 Documentation. Available at: <https://pytorch.org/>
- 36) Qiu, Q. et al. (2023) ‘Fine-scale urban land-cover mapping using Sentinel-2 and deep learning’, *GIScience & Remote Sensing*, 60(1), pp. 663–682.
- 37) Rahman, M. and Wang, Y. (2020) ‘Optimising IoU for segmentation tasks’, *ICIP*, pp. 198–205.
- 38) Rasterio Developers (2023) Rasterio Documentation. Available at: <https://rasterio.readthedocs.io/en/stable/>
- 39) Scikit-Learn Developers (2024) Scikit-Learn API Reference. Available at: <https://scikit-learn.org/>
- 40) Stehman, S. et al. (2021) ‘Validation of 10-m land-cover datasets’, *Remote Sensing of Environment*, 253, 112215.
- 41) Tsendbazar, N. et al. (2021) ‘Progress in global land-cover accuracy assessments’, *ISPRS Journal of Photogrammetry and Remote Sensing*, 173, pp. 74–92.
- 42) Wang, S. et al. (2023) ‘Class imbalance correction in segmentation models’, *Neural Computing & Applications*, 35, pp. 18893–18912.
- 43) Wang, X. et al. (2023) ‘A U-Net variant for land-cover classification under label noise’, *Remote Sensing*, 15(2), 322.
- 44) Zanaga, D. et al. (2021) ‘ESA WorldCover 2020 10m: Global land cover mapping’, *Earth System Science Data*, 13, pp. 3927–3950.

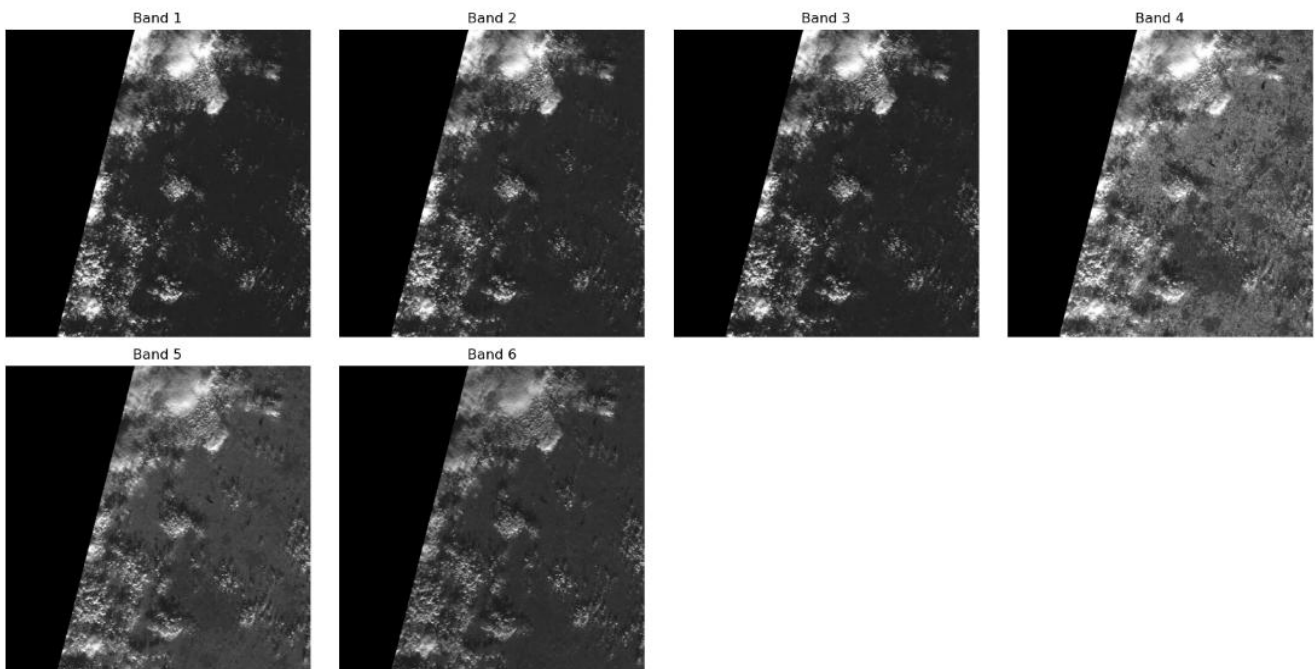
APPENDICES

Appendix 1:

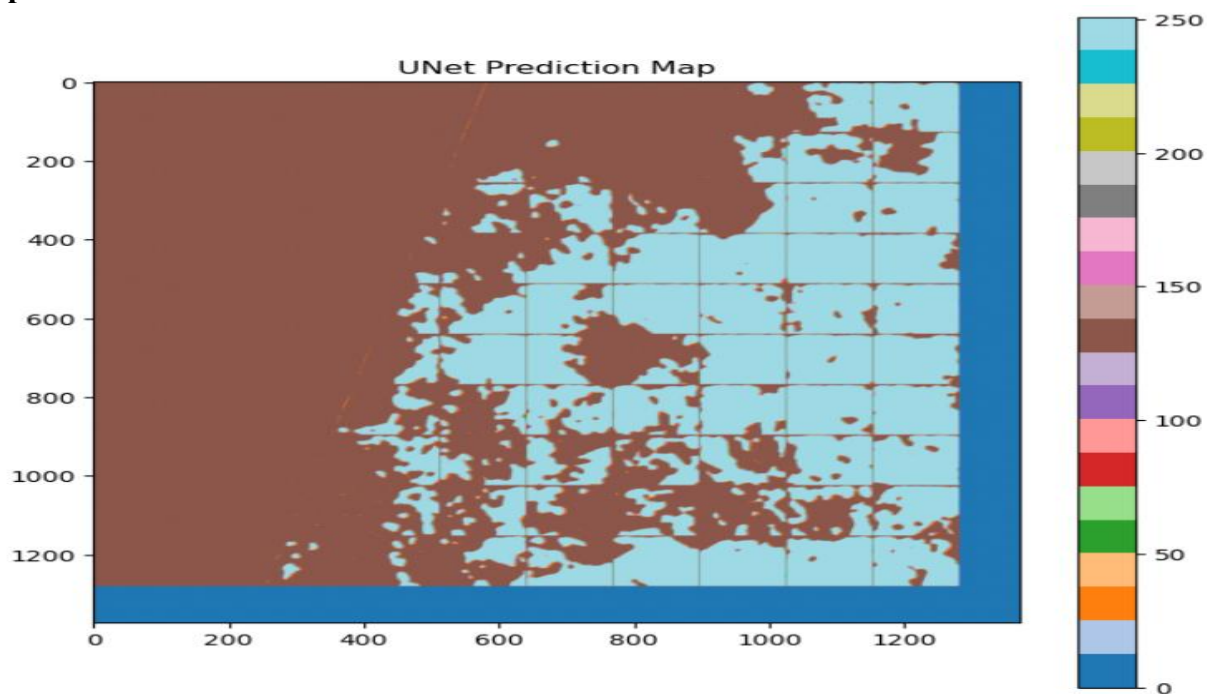
Sentinel-2 RGB Preview



Appendix 2:



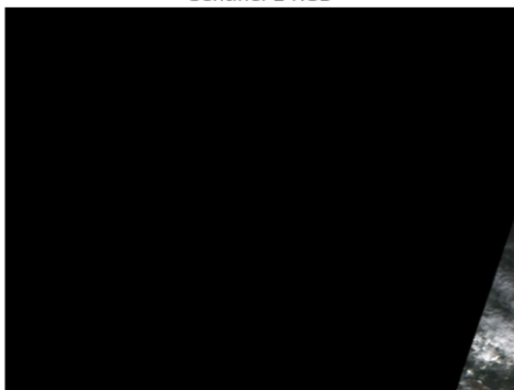
Appendix 3:



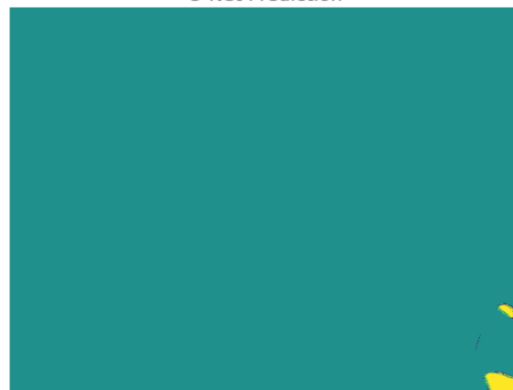
Appendix 4:

▲ Ground truth NOT loaded. Skipping that plot.

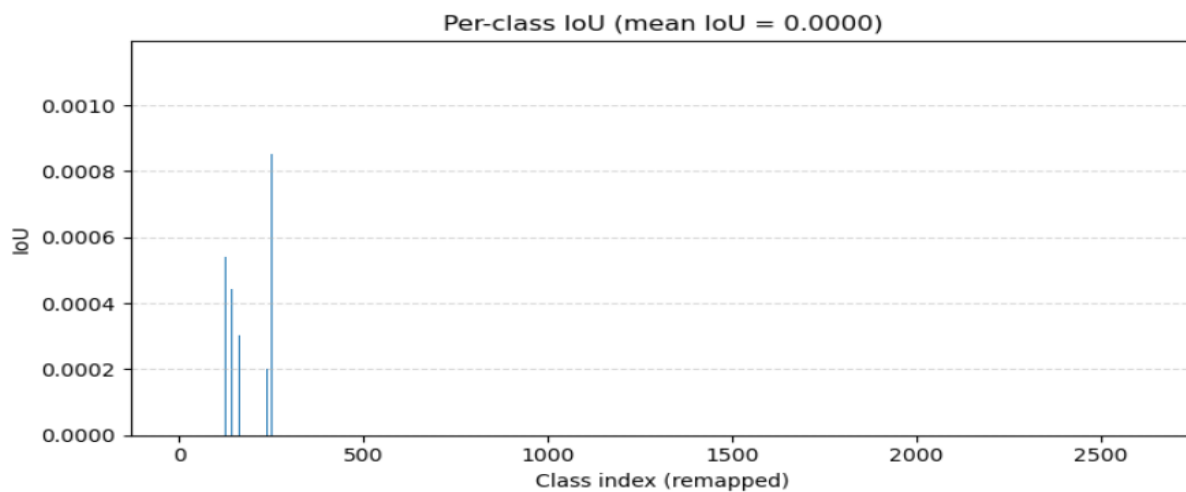
Sentinel-2 RGB



U-Net Prediction



Appendix 5:



Appendix 6: Importing Libraries

```
import os
import glob
import json
import random
from typing import List, Tuple, Dict

import numpy as np
import rasterio
from rasterio.enums import Resampling
from rasterio.vrt import WarpedVRT
from affine import Affine

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score, jaccard_score, classification_report
import joblib
```

Appendix 7: Configuration and Reproducibility

```
# CONFIGURATION
CONFIG = {
    "safe_dir": r"C:\Users\ASUS\Desktop\Desertation\S2A_MSIL2A_20251128T111431_N0511_R137_T30UMD_20251128T132409.SAFE",
    "label_tif": r"C:\Users\ASUS\Desktop\Desertation\ESA_WorldCover_10m_2021_v200_N52W003_S2RGBNIR.tif",
    "out_dir": r"C:\Users\ASUS\Desktop\Desertation\Outputs\output_run1",

    # Sentinel-2 band patterns
    "bands_10m": [
        "B02_10m.jp2", # Blue
        "B03_10m.jp2", # Green
        "B04_10m.jp2", # Red
        "B08_10m.jp2", # NIR
    ],
    "bands_20m": [
        "B11_20m.jp2", # SWIR1
        "B12_20m.jp2", # SWIR2
    ],

    # Spatial downsampling factor (for memory safety)
    # 1.0 -> full 10m resolution (heavy)
    # 0.5 -> ~20m
    # 0.25 -> ~40m (recommended if RAM is limited)
    "scale": 0.125,

    # Patching & training
    "patch_size": 128,
    "train_val_split": 0.8,
    "batch_size": 1, # Lower if you get CUDA OOM
    "num_epochs": 5, # start small, you can increase later
    "learning_rate": 1e-3,

    # WorldCover classes (11, class 0 = unlabeled)
    "num_classes": 11,

    # Device
    "device": "cuda" if torch.cuda.is_available() else "cpu",

    # Misc
    "seed": 42,
    "save_checkpoint_every": 5,
}

os.makedirs(CONFIG["out_dir"], exist_ok=True)
random.seed(CONFIG["seed"])
np.random.seed(CONFIG["seed"])
torch.manual_seed(CONFIG["seed"])
```

Appendix 8: Band Discovery, Reading, and Stacking

```

# UTILITIES - FIND BANDS
def find_band_files(safe_dir: str, patterns: List[str]) -> Dict[str, str]:
    matches = {}

    for pattern in patterns:
        # 1) Try the pattern as given
        search1 = os.path.join(safe_dir, "**", pattern)
        found = glob.glob(search1, recursive=True)

        # 2) Try without "_10m"/"_20m"
        if not found and "_10m" in pattern:
            alt = pattern.replace("_10m", "")
            search2 = os.path.join(safe_dir, "**", alt)
            found = glob.glob(search2, recursive=True)

        if not found and "_20m" in pattern:
            alt = pattern.replace("_20m", "")
            search2 = os.path.join(safe_dir, "**", alt)
            found = glob.glob(search2, recursive=True)

        # 3) Try generic "B0X".jp2
        if not found and "B0" in pattern:
            band_id = pattern.split("B0")[-1].split("_")[0] # e.g. "2" from "B02_10m.jp2"
            generic = f"B0{band_id}.jp2"
            search3 = os.path.join(safe_dir, "**", generic)
            found = glob.glob(search3, recursive=True)

        if not found:
            # Debug: show what's actually inside the SAFE
            all_jp2 = glob.glob(os.path.join(safe_dir, "**", "*.jp2"), recursive=True)
            print(f"\n[DEBUG] No files found for pattern '{pattern}' in {safe_dir}")
            print(f"[DEBUG] Example JP2 files found in SAFE:")
            for p in all_jp2[:10]:
                print(" ", p)
            raise FileNotFoundError(f"No files found for pattern {pattern} in {safe_dir}")

        matches[pattern] = found[0]

    return matches

# BAND READING, RESAMPLING, STACKING
def read_band_as_array(path: str, scale: float = 0.25) -> Tuple[np.ndarray, dict]:
    with rasterio.open(path) as src:
        if scale == 1.0:
            arr = src.read(1).astype(np.float32)
            profile = src.profile.copy()
        else:
            new_height = int(src.height * scale)
            new_width = int(src.width * scale)

            arr = src.read(
                1,
                out_shape=(new_height, new_width),
                resampling=Resampling.bilinear,
            ).astype(np.float32)

            scale_x = src.width / new_width
            scale_y = src.height / new_height
            new_transform = src.transform * Affine.scale(scale_x, scale_y)

            profile = src.profile.copy()
            profile.update(
                height=new_height,
                width=new_width,
                transform=new_transform,
            )

    return arr, profile

def resample_to_match(src_path: str, target_profile: dict) -> np.ndarray:
    with rasterio.open(src_path) as src:
        params = dict(
            crs=target_profile["crs"],
            transform=target_profile["transform"],
            width=target_profile["width"],
            height=target_profile["height"],
            resampling=Resampling.bilinear,
        )
        with WarpedVRT(src, **params) as vrt:
            arr = vrt.read(1).astype(np.float32)
    return arr

def stack_bands(b10_paths: List[str], b20_paths: List[str], out_path: str, scale: float) -> Tuple[np.ndarray, dict]:
    s the in-memory array as float16 (half precision)

    band_arrays = []
    base_profile = None

    # 1) Load 10m bands (base geometry)
    for p in b10_paths:
        arr, prof = read_band_as_array(p, scale=scale)
        band_arrays.append(arr)
        if base_profile is None:
            base_profile = prof

    # 2) Load & resample 20m bands to the same grid
    for p in b20_paths:
        if p:
            arr = resample_to_match(p, base_profile)
            band_arrays.append(arr)

    # 3) Stack into single array (C, H, W) in float16 to save RAM
    stacked = np.stack(band_arrays, axis=0).astype(np.float16)
    del band_arrays # free list memory ASAP

    # 4) Normalize in-place to [0,1]
    stacked /= 10000.0
    np.clip(stacked, 0.0, 1.0, out=stacked)

    # 5) Prepare metadata for GeoTIFF
    # We still write as float32 on disk; GDAL handles the cast.
    meta = base_profile.copy()
    meta.update(
        driver="Gtiff",
        count=stacked.shape[0],
        dtype="float32", # file on disk
        compress="lzw",
    )

    # 6) Write GeoTIFF (will internally cast float16 -> float32 in chunks)
    with rasterio.open(out_path, "w", **meta) as dst:
        dst.write(stacked.astype(np.float32))

    return stacked, meta

```

Appendix 9: Label Alignment and Class Remapping

```

# LABEL ALIGNMENT
def load_label_and_align(label_tif: str, ref_meta: dict) -> Tuple[np.ndarray, dict]:

    with rasterio.open(label_tif) as src:
        params = dict(
            crs=ref_meta["crs"],
            transform=ref_meta["transform"],
            width=ref_meta["width"],
            height=ref_meta["height"],
            resampling=Resampling.nearest,
        )
        with WarpedVRT(src, **params) as vrt:
            label = vrt.read(1).astype(np.int16)
            out_profile = ref_meta.copy()
            out_profile.update(count=1, dtype="int16")
    return label, out_profile

```

Appendix 10: Patch Extraction and Dataset Construction

```
# PATCHES & DATASET
def make_patches(
    image: np.ndarray,
    label: np.ndarray,
    patch_size: int = 256,
    stride: int = None,
    min_valid_fraction: float = 0.01,
) -> List[Tuple[np.ndarray, np.ndarray]]:

    C, H, W = image.shape
    if stride is None:
        stride = patch_size

    patches = []
    total = 0
    kept = 0

    for r in range(0, H - patch_size + 1, stride):
        for c in range(0, W - patch_size + 1, stride):
            total += 1
            img_patch = image[:, r:r+patch_size, c:c+patch_size]
            lbl_patch = label[r:r+patch_size, c:c+patch_size]

            valid_frac = np.count_nonzero(lbl_patch) / (patch_size * patch_size)
            if valid_frac >= min_valid_fraction:
                patches.append((img_patch, lbl_patch))
                kept += 1

    print(f" [make_patches] total windows={total}, kept={kept}, min_valid_fraction={min_valid_fraction}")
    return patches


class PatchDataset(Dataset):
    |

    def __init__(self, patches: List[Tuple[np.ndarray, np.ndarray]], augment: bool = False):
        self.patches = patches
        self.augment = augment

    def __len__(self) -> int:
        return len(self.patches)

    def __getitem__(self, idx: int) -> Tuple[torch.Tensor, torch.Tensor]:
        img, lbl = self.patches[idx]

        # Simple augment: flips & rotations
        if self.augment:
            if random.random() > 0.5:
                img = np.flip(img, axis=2).copy()
                lbl = np.flip(lbl, axis=1).copy()
            if random.random() > 0.5:
                img = np.flip(img, axis=1).copy()
                lbl = np.flip(lbl, axis=0).copy()
            k = random.randint(0, 3)
            if k:
                img = np.rot90(img, k, axes=(1, 2)).copy()
                lbl = np.rot90(lbl, k).copy()

        img_t = torch.tensor(img, dtype=torch.float32)
        lbl_t = torch.tensor(lbl, dtype=torch.long)
        return img_t, lbl_t
```

Appendix 11 : U-Net Model Architecture

U-Net Model

```
class DoubleConv(nn.Module):
```

```
    def __init__(self, in_ch: int, out_ch: int):
        super().__init__()
        self.conv_block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )
```

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.conv_block(x)
```

```
class UNetSmall(nn.Module):
```

```
    def __init__(self, in_channels: int, out_channels: int):
        super().__init__()
```

Encoder

```
    self.dconv_down1 = DoubleConv(in_channels, 16)
    self.pool1 = nn.MaxPool2d(2)
```

```
    self.dconv_down2 = DoubleConv(16, 32)
    self.pool2 = nn.MaxPool2d(2)
```

Bridge

```
    self.bridge = DoubleConv(32, 64)
```

Decoder

```
    self.up2 = nn.ConvTranspose2d(64, 32, 2, stride=2)
    self.dconv_up2 = DoubleConv(32 + 32, 32) # 32 up + 32 skip
```

```
    self.up1 = nn.ConvTranspose2d(32, 16, 2, stride=2)
    self.dconv_up1 = DoubleConv(16 + 16, 16) # 16 up + 16 skip
```

Output

```
    self.conv_last = nn.Conv2d(16, out_channels, 1)
```

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
```

Encoder

```
    c1 = self.dconv_down1(x)
    p1 = self.pool1(c1)
    c2 = self.dconv_down2(p1)
    p2 = self.pool2(c2)
```

Bridge

```

b = self.bridge(p2)

# Decoder
u2 = self.up2(b)
u2 = torch.cat([u2, c2], dim=1)
c_up2 = self.dconv_up2(u2)

u1 = self.up1(c_up2)
u1 = torch.cat([u1, c1], dim=1)
c_up1 = self.dconv_up1(u1)

out = self.conv_last(c_up1)
return out

```

Appendix 12: Training Procedure and Loss / Accuracy Monitoring

METRICS & TRAINING

```

def compute_metrics(y_true: np.ndarray, y_pred: np.ndarray, num_classes: int) -> dict:
    labels = list(range(1, num_classes + 1))

    per_class_iou = jaccard_score(
        y_true, y_pred, average=None, labels=labels, zero_division=0
    )
    mean_iou = float(np.nanmean(per_class_iou))
    f1_macro = f1_score(
        y_true, y_pred, average="macro", labels=labels, zero_division=0
    )

    return {
        "per_class_iou": per_class_iou.tolist(),
        "mean_iou": mean_iou,
        "f1_macro": f1_macro,
    }

def train_model(model: nn.Module, train_loader: DataLoader, val_loader: DataLoader, cfg: dict) -> nn.Module:
    device = cfg["device"]
    model = model.to(device)

    optimizer = torch.optim.Adam(model.parameters(), lr=cfg["learning_rate"])
    criterion = nn.CrossEntropyLoss(ignore_index=0)

    best_val_iou = -1.0
    best_model_path = os.path.join(cfg["out_dir"], "best_unet.pth")

    for epoch in range(cfg["num_epochs"]):
        model.train()
        running_loss = 0.0

        for imgs, masks in train_loader:
            imgs = imgs.to(device)
            masks = masks.to(device)

            optimizer.zero_grad()
            outputs = model(imgs)

```

```

    loss = criterion(outputs, masks)
    loss.backward()
    optimizer.step()
    running_loss += loss.item()

# Validation
model.eval()
preds_list = []
gts_list = []

with torch.no_grad():
    for imgs, masks in val_loader:
        imgs = imgs.to(device)
        outputs = model(imgs)
        preds = outputs.argmax(dim=1).cpu().numpy()
        gts = masks.numpy()
        preds_list.append(preds.reshape(-1))
        gts_list.append(gts.reshape(-1))

preds_flat = np.concatenate(preds_list)
gts_flat = np.concatenate(gts_list)

mask_valid = gts_flat > 0
preds_flat = preds_flat[mask_valid]
gts_flat = gts_flat[mask_valid]

metrics = compute_metrics(gts_flat, preds_flat, cfg["num_classes"])
mean_iou = metrics["mean_iou"]

print(
    f"Epoch {epoch+1}/{cfg['num_epochs']}: "
    f"loss={running_loss:.4f}, mIoU={mean_iou:.4f}, F1={metrics['f1_macro']:.4f}"
)

if (epoch + 1) % cfg["save_checkpoint_every"] == 0:
    ckpt_path = os.path.join(
        cfg["out_dir"], f"UNET_epoch{epoch+1}_mIoU{mean_iou:.4f}.pth"
    )
    torch.save(model.state_dict(), ckpt_path)
    print(f" Saved checkpoint: {ckpt_path}")

if mean_iou > best_val_iou:
    best_val_iou = mean_iou
    torch.save(model.state_dict(), best_model_path)
    print(f" *** New best model saved: {best_model_path}")

print(f"Best validation mIoU: {best_val_iou:.4f}")
return model

```

Appendix 13: Random Forest Baseline

```

# Random Forest Baseline
def train_random_forest(
    image: np.ndarray,
    label: np.ndarray,

```

```

    max_samples: int = 3000,
    n_estimators: int = 20,
) -> Tuple[RandomForestClassifier, str]:
    C, H, W = image.shape
    X = image.reshape(C, -1).T      # (H*W, C)
    y = label.flatten()            # (H*W,)

    # Use only labeled pixels (>0)
    mask = y > 0
    X_lab = X[mask]
    y_lab = y[mask]

    if len(y_lab) == 0:
        raise RuntimeError("No labeled pixels available for Random Forest.")

    # ◇ Cap number of samples to avoid memory issues
    n = min(max_samples, len(y_lab))
    idx_train = np.random.choice(len(y_lab), size=n, replace=False)
    X_train = X_lab[idx_train]
    y_train = y_lab[idx_train]

    # ◇ Very small RF to save memory
    rf = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=15,
        min_samples_leaf=5,
        n_jobs=1,          # 1 thread → less RAM overhead
        random_state=42,
    )

    rf.fit(X_train, y_train)

    # Simple test on another random subset (or same if few samples)
    idx_test = np.random.choice(len(y_lab), size=n, replace=False)
    X_test = X_lab[idx_test]
    y_test = y_lab[idx_test]
    y_pred = rf.predict(X_test)

    report = classification_report(y_test, y_pred, zero_division=0)
    return rf, report

```

Appendix 14: Full-Scene Inference and Predicted Map Generation

```

# FULL SCENE INFERENCE
def predict_whole_raster(
    model: nn.Module,
    stacked_arr: np.ndarray,
    patch_size: int,
    device: str,
) -> np.ndarray:
    C, H, W = stacked_arr.shape
    out = np.zeros((H, W), dtype=np.uint8)

```

```

model = model.to(device)
model.eval()

step = patch_size

with torch.no_grad():
    for r in range(0, H - patch_size + 1, step):
        for c in range(0, W - patch_size + 1, step):
            patch = stacked_arr[:, r:r+patch_size, c:c+patch_size]
            inp = torch.tensor(patch, dtype=torch.float32).unsqueeze(0).to(device)
            logits = model(inp)
            pred = logits.argmax(dim=1).cpu().numpy()[0]
            out[r:r+patch_size, c:c+patch_size] = pred

return out

```

Appendix 15: Main Function

```

def remap_worldcover_labels(labels: np.ndarray) -> tuple[np.ndarray, int]:
    unique_vals, counts = np.unique(labels, return_counts=True)
    nonzero_vals = unique_vals[unique_vals > 0]

    print(" [remap] unique non-zero label values:", nonzero_vals)

    if len(nonzero_vals) == 0:
        print(" [remap] WARNING: no non-zero labels found in this tile.")
        # everything will stay 0
        return np.zeros_like(labels, dtype=np.int64), 0

    # Map each distinct non-zero value to 1..N
    out = np.zeros_like(labels, dtype=np.int64)
    mapping = {}
    for new_idx, val in enumerate(nonzero_vals, start=1):
        out[labels == val] = new_idx
        mapping[val] = new_idx

    print(" [remap] mapping used (raw_value -> new_class):", mapping)
    n_classes = len(nonzero_vals)
    return out, n_classes

```

MAIN PIPELINE

```

def main(cfg: dict):
    print("=== Sentinel-2 Land Cover Pipeline: START ===")

    os.makedirs(cfg["out_dir"], exist_ok=True)

    # 1) Locate band files
    print("[1/9] Locating Sentinel-2 bands...")
    b10 = find_band_files(cfg["safe_dir"], cfg["bands_10m"])
    b20 = find_band_files(cfg["safe_dir"], cfg["bands_20m"])

    print(" 10m bands:")
    for pat, p in b10.items():

```

```

    print(f" {pat} -> {p}")
print(" 20m bands:")
for pat, p in b20.items():
    print(f" {pat} -> {p}")

# 2) Stack bands
print("[2/9] Stacking bands...")
stack_path = os.path.join(cfg["out_dir"], "sentinel2_stack.tif")
stacked, meta = stack_bands(list(b10.values()), list(b20.values()), stack_path, cfg["scale"])
print(f" Stacked image: {stack_path}")
print(f" Shape: {stacked.shape}") # (C, H, W)

# 3) Align labels
print("[3/9] Aligning labels...")
labels, label_meta = load_label_and_align(cfg["label_tif"], meta)
print(f" Label shape (raw): {labels.shape}")

C, H, W = stacked.shape
assert labels.shape == (H, W), f"Shape mismatch: image {(H, W)} vs label {labels.shape}"

labels, n_classes = remap_worldcover_labels(labels)

if n_classes == 0:
    raise RuntimeError("Remapped labels have no non-zero classes. Check that label_tif overlaps Sentinel-2 tile.")

CONFIG["num_classes"] = n_classes
print(f" [info] Number of classes after remap: {CONFIG['num_classes']}")

total_pixels = labels.size
labeled_pixels = np.count_nonzero(labels)
labeled_ratio = labeled_pixels / total_pixels
print(f" Labeled pixels after remap: {labeled_pixels} / {total_pixels} ({labeled_ratio:.4%})")

# 4) Generate patches
print("[4/9] Generating patches...")
patches = make_patches(
    stacked,
    labels,
    patch_size=cfg["patch_size"],
    stride=cfg["patch_size"],
    min_valid_fraction=0.01,
)
print(f" Total patches: {len(patches)}")
if len(patches) == 0:
    raise RuntimeError("No patches generated. Check labels/tile overlap.")

max_patches = 200
if len(patches) > max_patches:
    patches = random.sample(patches, max_patches)
    print(f" [info] Subsampled patches to {len(patches)} for memory reasons.")

# 5) Train/val split
print("[5/9] Splitting train/val...")

```

```

random.shuffle(patches)
split_idx = int(len(patches) * cfg["train_val_split"])
train_patches = patches[:split_idx]
val_patches = patches[split_idx:]
print(f" Train patches: {len(train_patches)}")
print(f" Val patches: {len(val_patches)}")

train_ds = PatchDataset(train_patches, augment=True)
val_ds = PatchDataset(val_patches, augment=False)

train_loader = DataLoader(train_ds, batch_size=cfg["batch_size"], shuffle=True, num_workers=0)
val_loader = DataLoader(val_ds, batch_size=cfg["batch_size"], shuffle=False, num_workers=0)

# 6) Train U-Net
print("[6/9] Training U-Net...")
model = UNetSmall(
    in_channels=stacked.shape[0],
    out_channels=cfg["num_classes"] + 1
)
model = train_model(model, train_loader, val_loader, cfg)

# 7) Random Forest baseline
print("[7/9] Training Random Forest baseline (lightweight)...")
rf, rf_report = train_random_forest(
    stacked,
    labels,
    max_samples=3000, # reduce to 1000 if RAM is still an issue
    n_estimators=20, # small forest to save memory
)

rf_path = os.path.join(cfg["out_dir"], "rf_baseline.joblib")
joblib.dump(rf, rf_path)
print(f" RF model saved: {rf_path}")

rf_report_path = os.path.join(cfg["out_dir"], "rf_report.txt")
with open(rf_report_path, "w") as f:
    f.write(rf_report)
print(f" RF report saved: {rf_report_path}")

# 8) Full-scene inference
print("[8/9] Running full-scene inference...")
pred_raster = predict_whole_raster(model, stacked, cfg["patch_size"], cfg["device"])
pred_meta = meta.copy()
pred_meta.update(driver="GTiff", count=1, dtype="uint8")
pred_path = os.path.join(cfg["out_dir"], "pred_unet.tif")
with rasterio.open(pred_path, "w", **pred_meta) as dst:
    dst.write(pred_raster.astype("uint8"), 1)
print(f" Prediction saved: {pred_path}")

# 9) Global metrics
print("[9/9] Computing global metrics...")
mask_valid = labels > 0

```

```

y_true = labels[mask_valid].flatten()
y_pred = pred_raster[mask_valid].flatten()
metrics = compute_metrics(y_true, y_pred, cfg["num_classes"])
print(" Metrics:")
print(json.dumps(metrics, indent=2))
metrics_path = os.path.join(cfg["out_dir"], "metrics.json")
with open(metrics_path, "w") as f:
    json.dump(metrics, f, indent=2)
print(f" Metrics saved: {metrics_path}")

print("=== Sentinel-2 Land Cover Pipeline: COMPLETED ===")

if __name__ == "__main__":
    main(CONFIG)

```